MusicMind: Stage 2 Implementation Report

EECS3311 Software Design, Fall 2026. Individual project.

Author	Sarina Kavoli (student ID 220816757)
Repository	https://github.com/sarinakavoli/EECS3311-MusicMind
Report written	November 3, 2026 (Stage 2 is due November 10, 2026)
Code state described	Branch `stage2-docs`, which holds the full implementation (CLI and GUI) plus the documentation. The GUI pull request (#4) and this documentation branch are not yet merged into `main` as of November 3.
Automated tests	846 passing in 51 test classes (a forced clean build on November 3), with no API keys and no database server.

> **Status of this document.** It describes what exists in the repository and what was actually run. Anything that was not run against the real Spotify and OpenAI accounts is marked as such. Items that only the author can fill in are shown like this: **[to be filled by Sarina]** (token and money costs, and every demonstration video link). Period 3 of the commit schedule (November 3-9) is in progress, so its commit count here is a count of what exists today.

Contents

1 Project overview and implementation status

2 Feature to implementation mapping (F1-F12 and the one new feature)

3 Implemented design patterns, with the updated UML diagrams

4 Changes and rationale (Original design, Implemented change, Reason, Impact)

5 AI tools and collaboration log

6 Development and verification summary (weekly commits, checks, defects, limitations)

7 Demonstration index

Supporting documents in the repository: `docs/STAGE1_CONFORMANCE.md` (feature to code, tests and live status), `docs/STAGE2_PLAN.md` (every change as Original, Change, Reason, Impact), `docs/AI_COLLABORATION_LOG.md`, `docs/DAILY_TEST_NOTES.md` (what was run live), `docs/GUI_TEST_FINDINGS.md`, `docs/diagrams/`. Class paths in this report are relative to `src/main/java/com/musicmind/`.

1 Project overview and implementation status

1.1 The application

MusicMind is an AI music-discovery and playlist-management agent built on the Spotify Web API. A listener types a goal in plain words, for example "a 90 minute study playlist, mostly familiar but with some new artists". MusicMind reads the listener's own Spotify listening data, turns the goal into a checked plan, retrieves candidate tracks, selects and orders them with its own deterministic recommender, validates the result against constraints (length, explicit content, blocked artists and tracks, duplicates), and creates or edits a real private playlist. It can also describe the listener's taste and how it changed, search in natural language, recommend new music, remember preferences and feedback, explain its choices, negotiate when a request cannot be met, and undo or restore earlier versions of a playlist.

Users. Individual Spotify listeners (Premium account required by Spotify for development apps) who want goal-driven playlists and controlled discovery, with a visible reason for every choice. Spotify development-mode apps allow at most five listed users without a quota extension.

1.2 Technology

Language and build	Java 17, Gradle 9.1 (wrapper committed). `./gradlew build` compiles and runs all tests; a Java 17 toolchain is downloaded automatically if missing.
Interfaces	A JavaFX 21 GUI (`./gradlew run`) and a command-line interface (`./gradlew runCli --args="help"`). Both call the same `AppController`; neither holds business logic.
Music data	Spotify Web API (Authorization Code with PKCE, redirect `http://127.0.0.1:8080/callback`). Spotify returns no artist genres to this app, so genres come from MusicBrainz and are cached by artist id.
Persistence	PostgreSQL through JDBC. Tests use an embedded PostgreSQL binary, so no server is needed to build.
LLM	One model, `gpt-4o` from OpenAI, behind the `LLMProvider` interface (`OpenAiProvider`). The test double `MockLLMProvider` exists only under `src/test`.

1.3 How the AI and LLM take part

The LLM is a planner and narrator; deterministic code decides and acts. It is called through `LLMProvider.complete` by these components only:

- `Planner`: turns a goal, a search text or an edit instruction into a structured, validated plan (`PlaylistSpec`, `AgentPlan`, `SearchQuery`), and words the options offered when a request cannot be met (F11).
- `ProfileBuilder` and `TrendComparator` prompts: one summary sentence for the taste profile and a short narrative for taste change.
- `ExplanationService`: a grounded explanation of a playlist, which `ResponseParser.verifyExplanation` checks against the recorded decision trace.
- `PlaylistEditor`: turns a free-text edit into commands (tracks to add always come from Spotify search, never from the model).
- `GoalFitReviewer`: a removal-only review of already retrieved tracks (see 1.5 and Section 4).

The LLM never names or adds a track. Selection, ordering, length and validation are computed. Every model reply passes through `ResponseParser`, and a failed or rejected reply falls back to a computed result that is labelled as computed. With no `LLM_API_KEY` the features that need the model say so, and F1, F2 and F3 use labelled fallbacks.

The agent behaviour is multi-step planning and tool use: `AgentSession.run()` moves through the states Idle, Planning, Retrieving, Selecting, Validating, Executing and Done (Failed on any error); `ToolManager` runs the Spotify and memory tools; `MemoryManager` stores preferences, feedback, sessions, snapshots and candidates.

1.4 Current working functionality

All twelve Stage 1 features (F1-F12) are implemented and reachable from both the CLI and the GUI. One feature was added that Stage 1 did not have (delete one of the listener's playlists, CLI only). No Stage 1 feature was dropped or replaced. Section 2 maps each one to its classes and methods; Section 6 records what was checked.

ID	Feature	Type in the code	Status
F1	Music Taste Profile	Hybrid	Modified
F2	Taste-Change Analysis	Hybrid	Modified
F3	Natural-Language Music Search	Hybrid	Modified
F4	Personalized Discovery	Deterministic	Modified
F5	AI Playlist Planning and Generation	Hybrid	Modified
F6	Natural-Language Playlist Modification	Hybrid	Modified
F7	Playlist Constraint Validation	Deterministic	Modified
F8	Preference Memory	Deterministic	Modified
F9	Recommendation Feedback	Deterministic	Implemented
F10	Grounded Recommendation Explanation	AI with a checked fallback	Modified
F11	Infeasible-Constraint Negotiation	Hybrid	Implemented
F12	Agent History and Snapshot/Restore	Deterministic	Modified
New	Delete a playlist	Deterministic	Added

"Modified" means the feature works as Stage 1 describes but with at least one documented difference (Section 4). The GUI is described as it is today: seven plain JavaFX tabs (Profile, Trends, Search, Discover, Playlists, Preferences, History), a Create playlist dialog and a conflict dialog.

1.5 Limitations and incomplete work (stated plainly)

The detail and the evidence for each point are in `docs/STAGE1_CONFORMANCE.md` (section 7) and `docs/DAILY_TEST_NOTES.md`.

- **Goal fit is unproven.** In the author's first GUI test (November 2) Create finished, but the songs for a "workout" goal did not suit it. Fixes were written afterwards (activity searches, a removal-only goal review, scoring changes), and **no playlist has been created since**, so it is not known that playlists now fit their goal.
- **Not run against the real accounts:** F11 with a real conflict (the prompt was only checked with stand-in text and in tests); delete (`delete`); recreating a deleted playlist from a snapshot; the GUI buttons Undo, Restore, Accept, Reject, Save and candidate controls pressed by a person (the same controller calls were run from the CLI for Undo and Restore); successful `feedback` and the other preference keys on the real account.
- **GUI redesign not done.** After the author's own user testing, a redesign (dark theme, a left-hand menu, clearer wording and a short description on every screen) is planned and has **not been started**. Known gaps are listed in `docs/GUI_TEST_FINDINGS.md`: no delete button, no chooser for blocking artists or songs, no candidate choice in the Create dialog, no artist names in track rows, plain styling.
- **Search by description is weak.** Spotify matches title words and this app has no tempo or mood data (Spotify provides no audio features to this app), so "mellow alternative for studying" returns poor results. Title and artist searches work.
- **Genre data is partial.** It depends on MusicBrainz; genres appeared for 13 of 20 top artists in the first live run, and many artists have none. Trends and discovery say when coverage is partial.
- **Policy correction (November 3).** The first version of the goal-fit review sent track titles and artist names to OpenAI. Spotify Developer Policy section III item 14 forbids ingesting Spotify Content into an AI model. It was changed the same day to send track ids and MusicBrainz genres only (details in Section 4). The review is now less precise, does not judge tracks with no known genre, and was not re-run live.
- **Open policy question (F6 edit prompt).** The prompt that turns an edit instruction into commands sends the model each track's id, explicit flag and length in minutes (no titles or names), because edits such as "15 minutes shorter" or "remove explicit" need them. Whether this counts as Spotify Content under the policy is unresolved and is for the author to decide.
- **Commit schedule.** Period 1 (October 20-26) has no code commits and missed its minimum; the single 5-point deduction therefore applies (Section 6).
- **Remaining work at the time of writing:** the full deterministic test suite and KUMA agent validation are Stage 3; for Stage 2, demonstration videos (links are placeholders in Section 7), cost figures (Section 5), merging the GUI and documentation branches, and the GUI redesign.

2 Feature to implementation mapping

Every Stage 1 feature (F1-F12) and the one new feature are mapped below with the fields the Stage 2 instructions require: feature, Stage 1 reference, status, classes and files with their responsibilities, key methods, design patterns and the demonstration video. Each entry then explains how the classes work together from user input to the final result, and says whether the feature is deterministic, AI-based or hybrid.

Common path for every feature: the CLI (`cli/CliApp`) or a GUI view (`app/view/*`) calls `controller/AppController`, which calls `controller/MusicAgentFacade` (Facade). The facade uses `tool/ToolManager` with `SpotifyTool` and `MemoryTool` to reach `music/MusicService` (implemented by `SpotifyAdapter` and wrapped by `RetryingMusicService` and `GenreEnrichingMusicService`) and `memory/MemoryManager` (repositories over PostgreSQL). These are not repeated in each entry unless the feature uses them in a particular way. "Video" cells are placeholders for the author; the full index is Section 7.

F01 Music Taste Profile

Feature ID and name	F01 Music Taste Profile (Stage 1 UC02, SD03)
Stage 1 reference	F1
Status	Modified (genres from MusicBrainz; stored profile; time range)
Classes / files and responsibilities	`AppController`, `MusicAgentFacade` (entry and orchestration); `tool/SpotifyTool` (tools `getTopItems`, `getSavedTracks`); `music/SpotifyAdapter` (Spotify HTTP and JSON); `music/GenreEnrichingMusicService` + `MusicBrainzAdapter` + `memory/repository/GenreRepository` (fill artist genres, cached); `agent/ProfileBuilder` (genre histogram, rankings, partial-profile notices); `agent/PromptBuilder` + `llm/OpenAiProvider` (summary sentence); `memory/MemoryManager` + `SessionRepository` (stored derived profile); `app/view/ProfileView`, `cli/CliApp` (display).
Key methods	`AppController.analyzeTaste`, `lastProfile`; `MusicAgentFacade.analyzeTaste`; `ProfileBuilder.build(TopItems, saved)`, `computeGenreHistogram`; `PromptBuilder.buildSummaryPrompt`; `MemoryManager.saveSession`.
Design patterns	Facade, Adapter, Decorator, Repository, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. The user presses "Refresh profile" (GUI) or runs `profile --refresh` (CLI). The controller asks the facade, which asks the tools for the top artists, top tracks and saved tracks; the decorators retry rate-limited calls and fill genres from MusicBrainz. `ProfileBuilder` computes the genre percentages and rankings and notes a partial profile. One LLM call words a one-sentence summary; with no key or a failed call a computed sentence is used and labelled "[computed from your listening data, not AI-written]". Only derived data (ids, genre counts, summary) is stored. **Type: hybrid.**

F02 Taste-Change Analysis

Feature ID and name	F02 Taste-Change Analysis (UC03, SD03)
Stage 1 reference	F2
Status	Modified (every step between ranges; genre-coverage caution)
Classes / files and responsibilities	`MusicAgentFacade` (reads the top artists of the three time ranges through `ToolManager`); `agent/TrendComparator` (set differences per range and between neighbouring ranges); `domain/TrendReport`, `RangeChange` (the result, including genre coverage per range); `PromptBuilder` + `OpenAiProvider` (the narrative); `app/view/TrendsView`, `cli/CliApp`.
Key methods	`AppController.analyzeTrends`; `MusicAgentFacade.analyzeTrends`; `TrendComparator.compare`; `TrendReport.steps`; `PromptBuilder.buildTrendPrompt`.
Design patterns	Facade, Adapter, Decorator, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. "Compare my listening" / `trends` loads the short, medium and long ranges. `TrendComparator` computes which artists and genres were added or dropped between each pair of ranges. A range with no data is marked and the others are still compared. When fewer than half of a range's artists have genre data (`trends.minGenreCoverage` = 0.5) a notice is added and the wording becomes cautious. The LLM words the narrative; a computed, labelled narrative is the fallback. **Type: hybrid.**

F03 Natural-Language Music Search

Feature ID and name	F03 Natural-Language Music Search (UC04, SD01)
Stage 1 reference	F3
Status	Modified (genre as a plain keyword; typed words merged; artists listed; stored candidates)
Classes / files and responsibilities	`agent/Planner` (asks the LLM for a `SearchQuery`); `agent/ResponseParser` (validates the reply, `verifyTracks` confirms every track exists in Spotify results); `domain/SearchQuery` (`toSpotifyQuery`); `agent/MoodGenreMap` (mood word to genre, from `mood-genres.json`); `agent/SearchRanker` (rank and merge results); `memory/repository/CandidateRepository` (track ids kept for the next playlist); `app/view/SearchView`, `cli/CliApp`.
Key methods	`AppController.searchMusic`, `addCandidate`, `removeCandidate`, `clearCandidates`, `getCandidates`; `MusicAgentFacade.searchMusic`, `withTypedWords`; `Planner.parseSearchQuery`; `SearchRanker.rank`, `merge`.
Design patterns	Facade, Adapter, Repository, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. The typed text goes to the LLM, which returns structured search terms (artists, activities, an era, a genre; mood words are removed and replaced by a mapped genre). The facade searches Spotify with that query and also with the words exactly as typed, then `SearchRanker.merge` puts exact title and artist matches first and removes duplicates of the same recording. Artists whose name the query contains are listed. If the LLM fails or has low confidence the text is searched as typed and the output says so. The user can add tracks to a stored candidate list that the next `create` starts with (unless `--no-candidates`). **Type: hybrid** (the LLM only reads the query; searching and ranking are deterministic).

F04 Personalized Discovery

Feature ID and name	F04 Personalized Discovery (UC05, SD04)
Stage 1 reference	F4
Status	Modified (never calls the LLM; known tracks always excluded)
Classes / files and responsibilities	`agent/RecommendationEngine` (Strategy context: retrieves pools, selects with the chosen strategy, scores); `agent/CandidateRetriever` (top, saved and recently played tracks, genre searches, artist to albums to tracks); `agent/strategy/StrategySelector` (picks Familiar, Discovery or Balanced from the novelty setting); `FamiliarStrategy`, `DiscoveryStrategy`, `BalancedStrategy` over `AbstractRecommendationStrategy`; `agent/PreferenceSettings` (stored preferences as F4 uses them); `app/view/DiscoveryView`, `FeedbackBar`, `WhyExpander`.
Key methods	`AppController.discoverSpec`, `discover`; `MusicAgentFacade.discover(profile, spec)`; `RecommendationEngine.retrieve`, `recommend`, `score`, `setStrategy`; strategies' `selectCandidates`, `scoreFamiliarity`, `excludeKnown`.
Design patterns	Strategy, Facade, Adapter, Repository, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. The facade loads the stored preferences and feedback, then `CandidateRetriever` builds candidate pools from the listener's own data (Spotify provides no recommendation endpoint to this app). `StrategySelector` sets a strategy on the engine, which excludes every track the listener already knows, drops blocked artists and tracks, and scores the rest (feedback boosts or penalises). Each result carries a reason built from recorded facts (known or new artist, where it was found, feedback). If there are too few candidates the genre set widens or novelty drops, and a notice says so. **Type: deterministic.** A test (`AppControllerDiscoverTest.discoverNeverCallsTheLlm`) checks that no model call happens.

F05 AI Playlist Planning and Generation

Feature ID and name	F05 AI Playlist Planning and Generation (UC06, SD01)
Stage 1 reference	F5
Status	Modified (goal-fit review; activity searches; optional candidates; rollback on failure)
Classes / files and responsibilities	`domain/AgentSession` (State context; `run()`); `agent/state/*State` (one class per phase) with `RunContext` and `RunServices`; `agent/Planner` (goal to plan, merging flags over goal over stored preferences); `agent/RecommendationEngine`, `CandidateRetriever`, `Retrieval`; `agent/GoalFitReviewer` (removal-only review); `agent/ConstraintChecker` (feasibility, F11); `agent/PlaylistValidator`; `agent/command/PlaylistManager` with `CreatePlaylistCommand` and `AddItemsCommand`; `agent/ExplanationService`; `app/view/CreatePlaylistDialog`.
Key methods	`AppController.createPlaylist(AgentRequest)`, `lastSession`, `setNegotiationHandler`, `setProgressListener`; `AgentSession.run`; `Planner.plan`; `RecommendationEngine.retrieve/recommend(Retrieval, ...)`; `GoalFitReviewer.review`, `reviewPicks`; `ConstraintChecker.checkFeasibility`; `PlaylistValidator.validate`; `PlaylistManager.executeAll`.
Design patterns	State, Command, Strategy, Facade, Adapter, Repository, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. The goal and any flags (`--minutes`, `--no-explicit`, `--familiar`, `--name`, `--no-candidates`) become an `AgentRequest`. `AgentSession.run()` then walks the states: *Planning* (the LLM parses the goal; the mood becomes genres through `MoodGenreMap`); *Retrieving* (one Spotify read shared by all later steps, the goal-fit review, then the feasibility check, which starts F11 if the request is infeasible); *Selecting* (user candidates first, then `BalancedStrategy`, then a review of the final picks); *Validating* (a violation sends the run back to selection, at most 3 times); *Executing* (`CreatePlaylistCommand` then `AddItemsCommand` through the Command invoker); *Done* (store the session and write the explanation). Any error goes to *Failed*, which rolls the new playlist back. An expired Spotify login is renewed once and the phase repeats. **Type: hybrid.** The LLM parses the goal, suggests activity search phrases, words the F11 options, writes the explanation and may remove already retrieved tracks that do not suit the goal; it never names or adds one.

F06 Natural-Language Playlist Modification

Feature ID and name	F06 Natural-Language Playlist Modification (UC07, SD02)
Stage 1 reference	F6
Status	Modified (editor class; snapshot before each change; undo across runs; playlist list)
Classes / files and responsibilities	`agent/PlaylistEditor` (instruction to an `EditPlan` of `CommandSpec`s; mood-only heuristic); `agent/ResponseParser.forPlaylist(...).parseCommands` (binds specs to real tracks); `agent/ConstraintChecker.checkEdit` (F11 for edits); `agent/command/PlaylistManager` (invoker: snapshot, batch, undo) with `AddItemsCommand`, `RemoveTrackCommand`, `ReplaceTrackCommand`, `ReorderCommand`, `BatchCommand`; `agent/PlaylistValidator`; `app/view/PlaylistView`.
Key methods	`AppController.modifyPlaylist`, `undo`, `getPlaylists`, `getPlaylist`; `MusicAgentFacade.modifyPlaylist`; `PlaylistEditor.plan`; `ConstraintChecker.checkEdit`; `PlaylistManager.modify`, `undo`; `PlaylistCommand.execute/undo/simulate`.
Design patterns	Command, Facade, Adapter, Repository, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. The user selects a playlist and types an instruction such as "make it 15 minutes shorter". The facade reads the playlist; `PlaylistEditor` asks the LLM for steps, and a feeling-only edit ("calmer") uses the genre heuristic and is flagged "Approximate". Tracks to add come from Spotify search. Before any write the edit is simulated: if it would add violations the editor re-plans (bounded), and if it would leave the playlist outside its length limits F11 offers a partial edit. `PlaylistManager.modify` saves a snapshot, then applies one undoable `BatchCommand`. The playlist is re-validated and a change summary is shown. `undo` reverses the last change, using the stored snapshot when the process is new (each CLI run is a new process). An ambiguous instruction produces a question and changes nothing. **Type: hybrid.**

F07 Playlist Constraint Validation

Feature ID and name	F07 Playlist Constraint Validation (UC08)
Stage 1 reference	F7
Status	Modified (blocked tracks added; blocks in force now; target length follows edits)
Classes / files and responsibilities	`agent/PlaylistValidator` (pure rule checks); `domain/ValidationResult` (pass or fail per rule and the offending track ids); `domain/PlaylistSpec` (the constraints); `controller/PlaylistTargets` (a playlist's target length over time); `app/view/PlaylistView` (badges through `ViewFormat.badges`).
Key methods	`AppController.validatePlaylist`, `lastValidationTargetMinutes`; `MusicAgentFacade.validatePlaylist`; `PlaylistValidator.validate`, `durationStatus`, `introducedViolations`; `ValidationResult.offending`.
Design patterns	Facade, MVC (the validator is also the check inside the State pattern's `ValidatingState`)
Video link / timestamp	**[link to be added by Sarina]**

How it works. `validate <id>` / the badge on the Playlists tab reads the playlist and judges it against the rules: explicit content, duplicates by id, target duration within tolerance, blocked artists, blocked tracks, and minimum and maximum track count. The spec saved with the creating run supplies the constraints, except that the blocks in force now always apply. The result lists which rule failed and which tracks offend; the same validator drives the adjustment loops in F5 and F6. **Type: deterministic** (no LLM).

F08 Preference Memory

Feature ID and name	F08 Preference Memory (UC09)
Stage 1 reference	F8
Status	Modified (artists typed by name stored as ids; choose-then-block calls; safe defaults)
Classes / files and responsibilities	`agent/PreferenceSettings` (validation and reading of the keys `blockedArtists`, `blockedTracks`, `explicitPolicy`, `familiarBias`, `preferredStyles`); `memory/MemoryManager` + `memory/repository/PreferenceRepository` (PostgreSQL); `tool/MemoryTool`; `app/view/PreferencesView`, `cli/CliApp`.
Key methods	`AppController.parsePreference`, `setPreference`, `getPreferences`, `displayPreference`, `findArtists`, `findTracks`, `blockArtist`, `blockTrack`, `savedFamiliarBias`; `MemoryManager.savePreference`, `loadPreferences`.
Design patterns	Repository, Facade, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. `prefs set <key> <value>` (or Save on the Preferences tab) is validated (for example `familiarBias` must be between 0 and 1), turned into ids (an artist typed by name is looked up and stored as a Spotify id; names are read for display and never stored) and saved by the repository. F4, F5 and F6 read these preferences. If the database is down the app keeps running with safe defaults and warns; blocking is refused when the stored preferences cannot be read so existing blocks are never overwritten. **Type: deterministic.**

F09 Recommendation Feedback

Feature ID and name	F09 Recommendation Feedback (UC10, SD04)
Stage 1 reference	F9
Status	Implemented (the track id is now validated before saving)
Classes / files and responsibilities	`AppController` (`requireTrackId` checks the id); `memory/MemoryManager` + `memory/repository/FeedbackRepository` (one verdict per user and track); `domain/Feedback`, `Verdict`; `agent/RecommendationEngine` (score boost or penalty); `app/view/FeedbackBar`.
Key methods	`AppController.recordFeedback`, `requireTrackId`; `MemoryManager.saveFeedback`; `RecommendationEngine.score`.
Design patterns	Repository, Facade, Strategy (the stored verdict changes later scores), MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. Accept or Reject on a row (or `feedback <trackId> accept|reject`) passes through `requireTrackId` (22 letters or digits, or a `spotify:track:` URI); a bad id saves nothing. The verdict is upserted by user and track, and later `RecommendationEngine.score` calls boost accepted and penalise rejected tracks. **Type: deterministic.**

F10 Grounded Recommendation Explanation

Feature ID and name	F10 Grounded Recommendation Explanation (UC11)
Stage 1 reference	F10
Status	Modified (stricter checks; titles shown from the run; explain by playlist id)
Classes / files and responsibilities	`agent/ExplanationService` (builds the explanation from a session); `domain/DecisionTrace` (strategy, per-track reasons, constraints met, requested genres, stored with the session); `agent/PromptBuilder.buildExplainPrompt`; `agent/ResponseParser.verifyExplanation` (rejects numbers, quoted names, genres or preference claims the trace does not record); `app/view/WhyExpander`.
Key methods	`AppController.explain` (a session id or a playlist id); `MusicAgentFacade.explain`; `ExplanationService.explain`; `ResponseParser.verifyExplanation`.
Design patterns	Facade, Repository, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. The explain request reads the decision trace stored by the creating run, gives only those recorded facts to the LLM, and then `verifyExplanation` checks the reply. A rejected or missing reply is replaced by a computed sentence labelled as such; a missing trace gives a general note. The prompt forbids saying that a track fits a mood or goal unless its own recorded reason says so. **Type: AI with a checked, computed fallback.**

F11 Infeasible-Constraint Negotiation

Feature ID and name	F11 Infeasible-Constraint Negotiation (UC12)
Stage 1 reference	F11
Status	Implemented (extra relaxation kinds; applies to create and to modify)
Classes / files and responsibilities	`agent/ConstraintChecker` (`checkFeasibility` on a `CandidatePool`, `checkEdit` on an edit); `domain/FeasibilityReport`, `Relaxation` (shorten, change the share, allow explicit, apply only part of an edit); `agent/Planner.phraseOptions` (the LLM words the options, used only if it returns one sentence per option) and `Planner.regenerate`; `agent/NegotiationHandler` (the choice callback); `agent/state/Negotiator` (one round); `app/view/NegotiationDialog`, the CLI prompt.
Key methods	`ConstraintChecker.checkFeasibility`, `checkEdit`; `Planner.phraseOptions`, `regenerate`; `NegotiationHandler.choose`.
Design patterns	State (it runs inside `RetrievingState`), Facade, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. When the eligible pool cannot satisfy the request (for example 600 minutes of only familiar tracks) or an edit would empty the playlist, the checker counts what is available and builds relaxations. The LLM words them; the CLI prompts or the GUI shows a dialog; the user's choice is applied (the plan is regenerated, or only part of the edit is applied). Cancelling writes nothing to Spotify. With no handler or no possible relaxation the run stops with an explanation. **Type: hybrid** (the feasibility check is computed; the LLM only words the options).

F12 Agent History and Snapshot/Restore

Feature ID and name	F12 Agent History and Snapshot/Restore (UC13, SD02)
Stage 1 reference	F12
Status	Modified (open one session; restore saves the current version first; recreate a missing playlist)
Classes / files and responsibilities	`memory/MemoryManager` with `SessionRepository` and `PlaylistHistoryRepository` (sessions, and snapshots that hold ids and the playlist name only); `agent/command/PlaylistManager` and `RestoreSnapshotCommand` (replay a snapshot as an undoable command); `domain/SnapshotInfo`; `app/view/HistoryView`.
Key methods	`AppController.getHistory`, `getSession`, `getSnapshots`, `restore`, `recreateFromSnapshot`; `MemoryManager.loadHistory`, `loadSnapshot`, `saveSnapshot`; `PlaylistManager.restoreFrom`.
Design patterns	Command, Repository, Facade, MVC
Video link / timestamp	**[link to be added by Sarina]**

How it works. `history` lists sessions newest first and the saved playlist versions; `history <id>` opens one session's request, plan and result. `restore <snapshotId>` first saves the playlist's current content (so the restore can be undone), then replays the snapshot through `RestoreSnapshotCommand` and re-reads titles from Spotify when needed. If the Spotify playlist no longer exists, `PlaylistMissingException` is raised and the interface offers to recreate it with `CreatePlaylistCommand`. A restore of an unchanged playlist says that nothing changed. **Type: deterministic.**

New: Delete a playlist

Feature ID and name	New: Delete a playlist (UC14, added in Stage 2)
Stage 1 reference	New
Status	Implemented (added). CLI only; no GUI control yet.
Classes / files and responsibilities	`agent/command/DeletePlaylistCommand` (calls `MusicService.unfollowPlaylist`, Spotify's only delete); `PlaylistManager.delete` (saves a snapshot first and removes nothing if it cannot); `MusicAgentFacade.deletePlaylist` (refuses a playlist not in the user's own list); `cli/CliApp` (asks for the playlist name to be typed).
Key methods	`AppController.deletePlaylist` (returns the snapshot id); `MusicAgentFacade.deletePlaylist`; `PlaylistManager.delete`; `MusicService.unfollowPlaylist`.
Design patterns	Command, Adapter, Facade
Video link / timestamp	**[link to be added by Sarina]**

How it works. `delete <playlistId>` checks that the playlist is the user's own, saves a snapshot, asks the user to type the playlist's name, and only then unfollows it. The command is not pushed on the undo stack because Spotify cannot restore a playlist under its old id; the way back is F12 (`restore` reports the playlist missing and offers to recreate it). **Type: deterministic.** *Not yet run against the real account.*

Interfaces

CLI (`cli/CliApp`): an instance built from a controller factory and input and output streams so tests can drive it. Commands: `login`, `help`, `exit`, `profile`, `trends`, `search`, `candidates`, `discover`, `create`, `playlists`, `show`, `modify`, `undo`, `validate`, `prefs`, `feedback`, `explain`, `history`, `restore`, `delete`. Exit status 0 means done, 1 could not complete, 2 wrong command or arguments. With no arguments it runs an interactive session. **GUI** (`app/GuiApp`, `app/view/*`): seven tabs, a Create playlist dialog and the F11 dialog. One background worker (`app/view/GuiContext.java`) runs every controller call, and opening the window contacts neither Spotify nor the browser.

3 Implemented design patterns

Five patterns are counted for the course: **Adapter, Strategy, Command, State and Facade**, as in Stage 1. MVC, Repository and Decorator are also used and are not counted. Each is explained below with its problem, participants, files and methods, and whether it matches Stage 1. Details by class are also in `docs/STAGE1_CONFORMANCE.md`, section 4.

3.1 Adapter

Problem. The core must not depend on the wire formats of Spotify, MusicBrainz or OpenAI, which change (Spotify removed endpoints in November 2024 and February 2026). **Matches Stage 1:** yes; the MusicBrainz adapter is new (R18).

Class / interface	Role	Responsibility
`music/MusicService`	Target	Stable operations the agent uses: `getTopArtists`, `search`, `createPlaylist`, `addItems`, `unfollowPlaylist`, `getUserPlaylists`, and so on.
`music/SpotifyAdapter`	Adapter	Translates each call into Spotify HTTP requests (PKCE sign-in through `OAuthTokenStore`) and maps JSON to `Track`, `Artist`, `Playlist`. Knows the February 2026 shapes (`/items`, 10 results per search).
`music/GenreService` / `MusicBrainzAdapter`	Target / Adapter	Looks up an artist's genres in MusicBrainz, rate-limited, with an outage pause.
`llm/LLMProvider` / `OpenAiProvider`	Target / Adapter	`complete(prompt)` hides the OpenAI chat API.
Agent classes, `ToolManager`	Clients	Depend only on the interfaces.

Why useful. All Spotify knowledge is in one class, and the tests replace Spotify with a local HTTP server (`SpotifyAdapterTest`) and the LLM with a mock without touching the agent.

3.2 Strategy

Problem. F4 and F5 need different mixes of familiar and new music, chosen at run time. **Matches Stage 1:** yes; `AbstractRecommendationStrategy.bindProfile(profile)` was added (R7) because the helper methods need the listener's profile.

Class / interface	Role	Responsibility
`agent/strategy/RecommendationStrategy`	Strategy	`selectCandidates(pools, spec)`.
`AbstractRecommendationStrategy`	Base	Shared helpers `scoreFamiliarity`, `excludeKnown`; the bound profile.
`FamiliarStrategy`, `DiscoveryStrategy`, `BalancedStrategy`	Concrete strategies	Prefer known artists; prefer new music; mix by ratio.
`agent/RecommendationEngine`	Context	Holds the strategy (`setStrategy`) and delegates selection; adds scoring and exclusion.
`agent/strategy/StrategySelector`	Chooser	Picks the strategy from the request or novelty setting.

Why useful. A new policy is a new class. `StrategyTest` and `StrategySelectorTest` test each in isolation. F4 uses the strategy the user's novelty setting selects; F5 uses `BalancedStrategy`.

3.3 Command

Problem. Playlist edits must be discrete, reversible, composable operations, recorded for history and undoable, including across separate CLI runs. **Matches Stage 1:** yes; three commands were added and undo also works from stored snapshots (R10).

Class / interface	Role	Responsibility
`agent/command/PlaylistCommand`	Command	`execute()`, `undo()`, plus `simulate`, `describe`, `playlistId`.
`CreatePlaylistCommand`, `AddItemsCommand`, `RemoveTrackCommand`, `ReplaceTrackCommand`, `ReorderCommand`	Concrete commands (Stage 1)	One operation each; removal and replacement re-add and reorder on undo.
`BatchCommand`, `RestoreSnapshotCommand`, `DeletePlaylistCommand`	Concrete commands (added)	One undo step for an edit; replay a saved version; unfollow a playlist.
`agent/command/PlaylistManager`	Invoker	Holds the history Deque; `execute`, `executeAll` (rolls back applied commands if one fails), `modify` (snapshot first), `undo`, `restoreFrom`, `delete`.
`music/MusicService`	Receiver	Performs the change on Spotify.

Why useful. F6 undo and F12 restore share one mechanism, and a failed multi-step change is undone instead of leaving a half-changed playlist.

3.4 State

Problem. A create run moves through ordered phases with different actions and failure handling in each. **Matches Stage 1:** yes; shared run data lives in `RunContext` and `RunServices`.

Class / interface	Role	Responsibility
`agent/state/SessionState`	State	`handle(session)`, `name()`, `isTerminal()`.
`IdleState`, `PlanningState`, `RetrievingState`, `SelectingState`, `ValidatingState`, `ExecutingState`, `DoneState`, `FailedState`	Concrete states	Each performs its phase and sets the next state; `FailedState` centralises rollback and reporting.
`domain/AgentSession`	Context	`setState`, `getState`, `run()` loops until a terminal state.
`RunContext`, `RunServices`, `Negotiator`	Support	Data and collaborators of one run; one F11 round.

Why useful. The flow is explicit and testable (`CreatePlaylistFlowTest` checks the lifecycle, bounded retries, token renewal and rollback), and the GUI shows the state names as progress.

3.5 Facade

Problem. The GUI and CLI need one simple entry point over many collaborators, without duplicating orchestration.
Matches Stage 1: yes; `AppController` adds the `last*()` accessors the interfaces need (R9).

Class	Role	Responsibility
`controller/MusicAgentFacade`	Facade	Coarse operations: `analyzeTaste`, `analyzeTrends`, `searchMusic`, `discover`, `createPlaylist`, `modifyPlaylist`, `validatePlaylist`, `explain`, `restore`, `deletePlaylist`.
`Planner`, `ToolManager`, `RecommendationEngine`, `PlaylistManager`, `PlaylistValidator`, `ConstraintChecker`, `MemoryManager`, `ExplanationService`	Subsystem	Coordinated by the facade.
`controller/AppController`	Client entry	The one class both interfaces call; it holds no business logic of its own beyond input checks.

3.6 Also used (not counted)

- **MVC.** Views (`app/view/*` and the CLI) call `AppController`; the domain classes are the model. Matches Stage 1.
- **Repository.** `Repository<T>` with `PreferenceRepository`, `FeedbackRepository`, `SessionRepository`, `PlaylistHistoryRepository`, `CandidateRepository` behind `MemoryManager`; `GenreRepository` caches genres. `CandidateRepository` and `GenreRepository` are new.
- **Decorator.** `RetryingMusicService` (backoff on HTTP 429; safe retries only) and `GenreEnrichingMusicService` (fills artist genres) wrap a `MusicService` without changing it. `GenreEnrichingMusicService` is new (R18).

3.7 Updated class diagram (application and agent architecture): overview

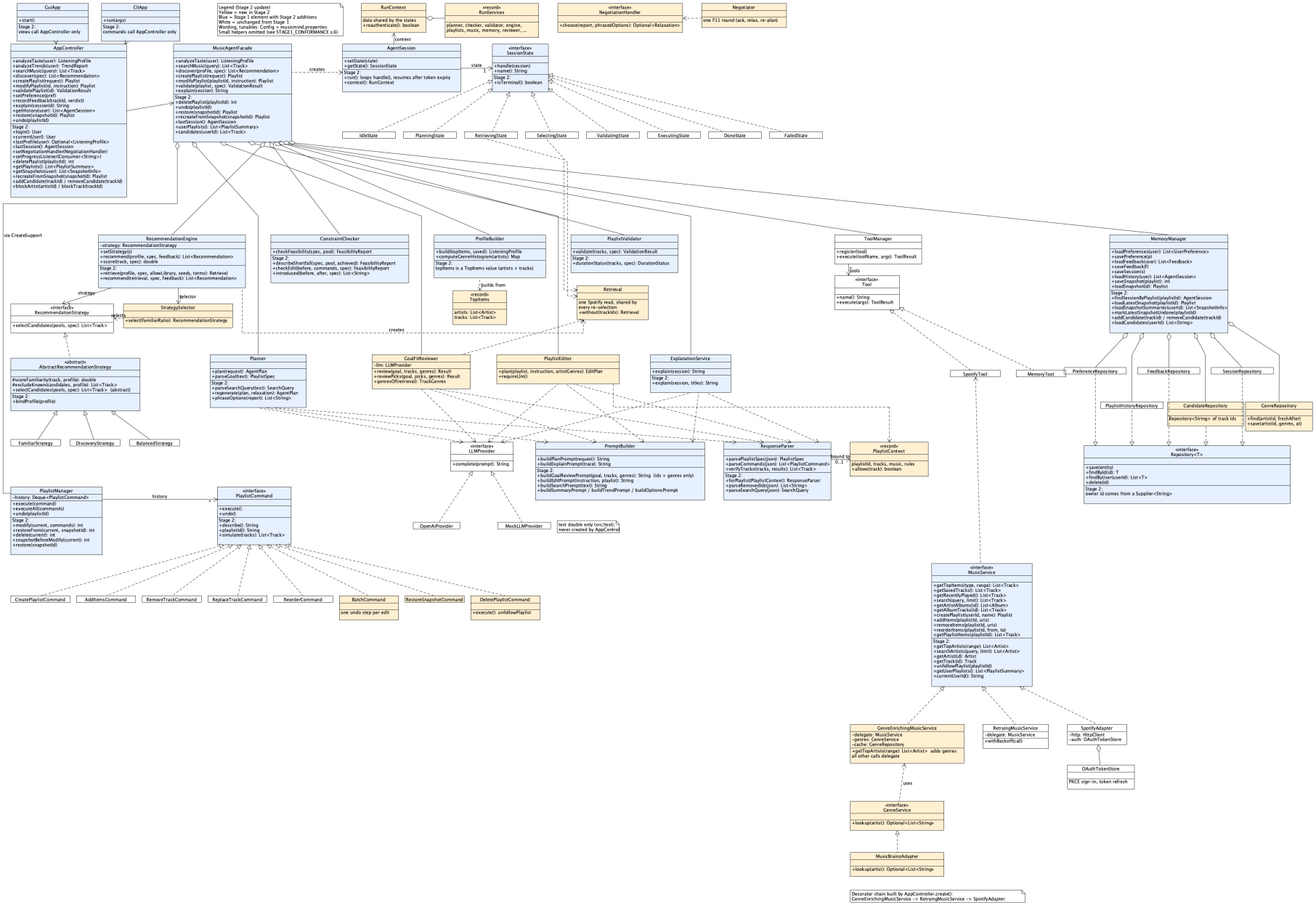


Figure 3.1. Overview of the whole diagram. It is too dense to read at this size; the next four pages show it in four readable quarters. Yellow = new in Stage 2; blue = a Stage 1 element with Stage 2 additions; white = unchanged. Small supporting classes are left out (`docs/STAGE1_CONFORMANCE.md`, section 6). Full PNG: `docs/diagrams/png/MusicMind_ClassArchitecture.png`; source: `docs/diagrams/uxf/`; Stage 1 version: `docs/diagrams/png/stage1/`.

3.7 Class diagram, quarter 1 of 4 (top left)

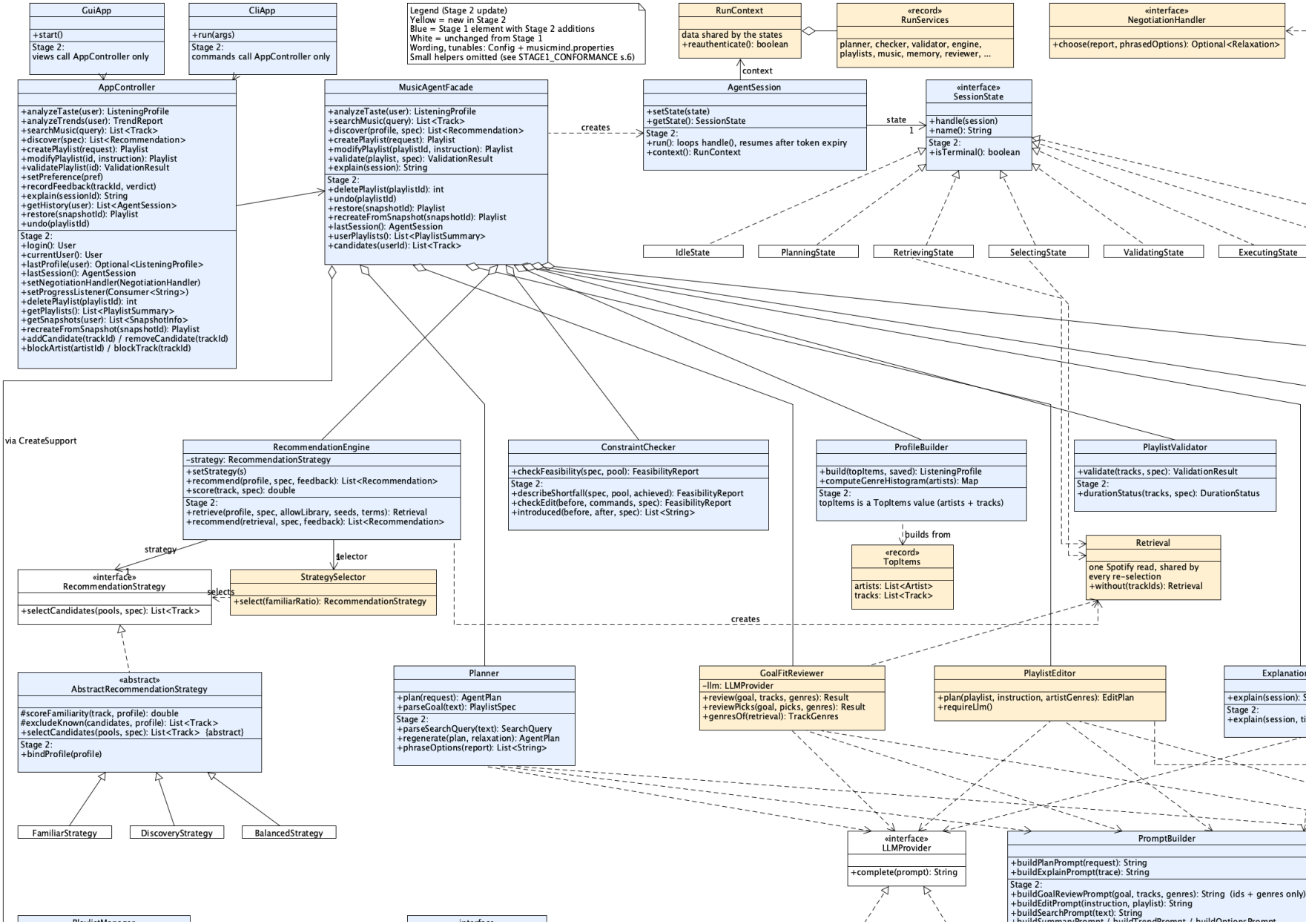


Figure 3.1 (continued). Crops overlap slightly. Colours as in the overview.

3.7 Class diagram, quarter 2 of 4 (top right)

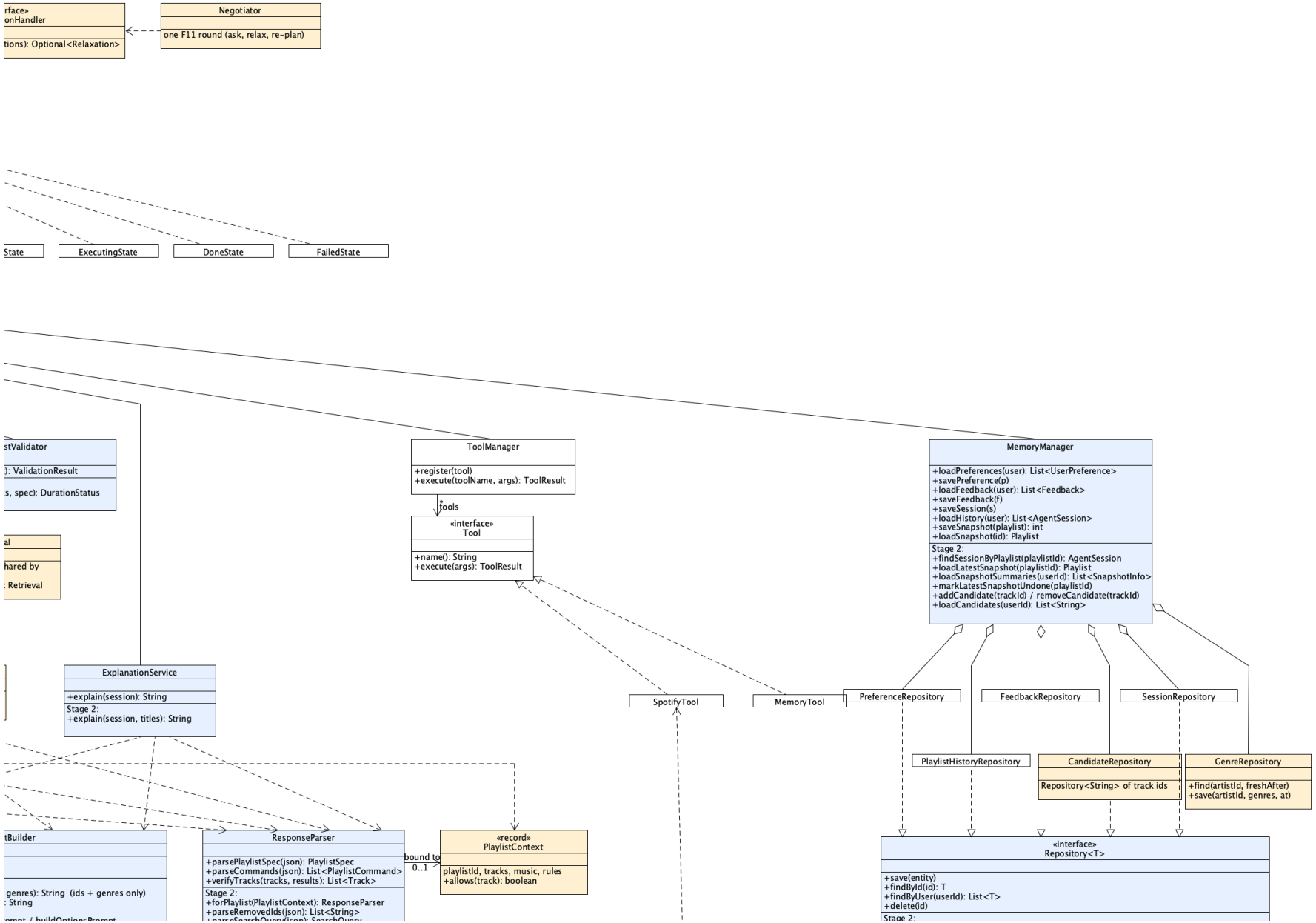


Figure 3.1 (continued). Crops overlap slightly. Colours as in the overview.

3.7 Class diagram, quarter 3 of 4 (bottom left)

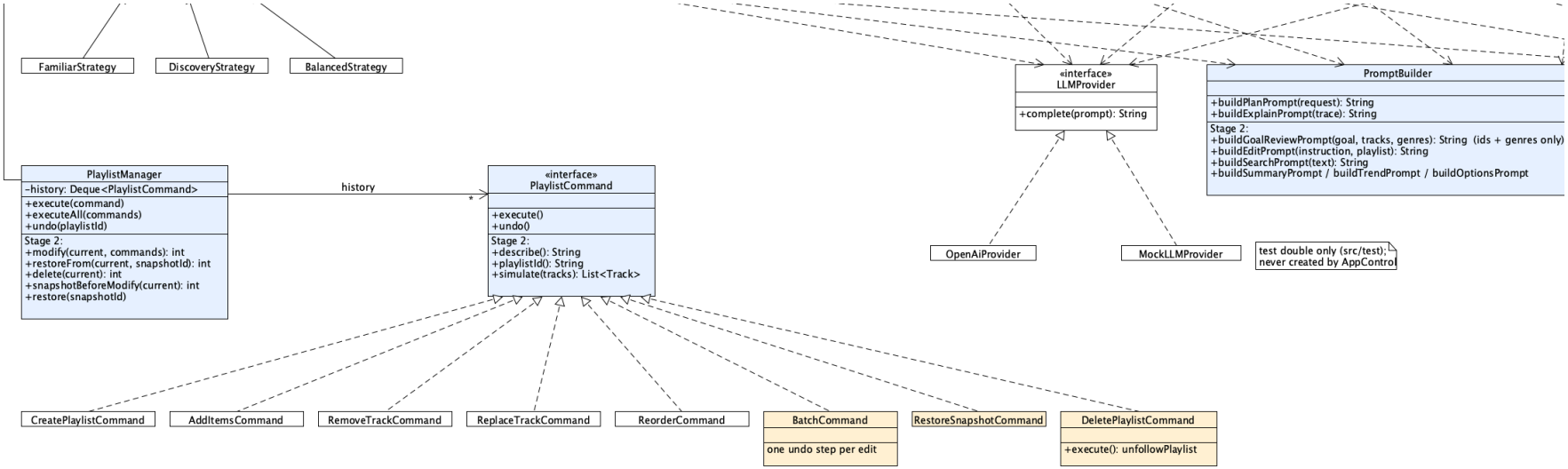


Figure 3.1 (continued). Crops overlap slightly. Colours as in the overview.

3.7 Class diagram, quarter 4 of 4 (bottom right)

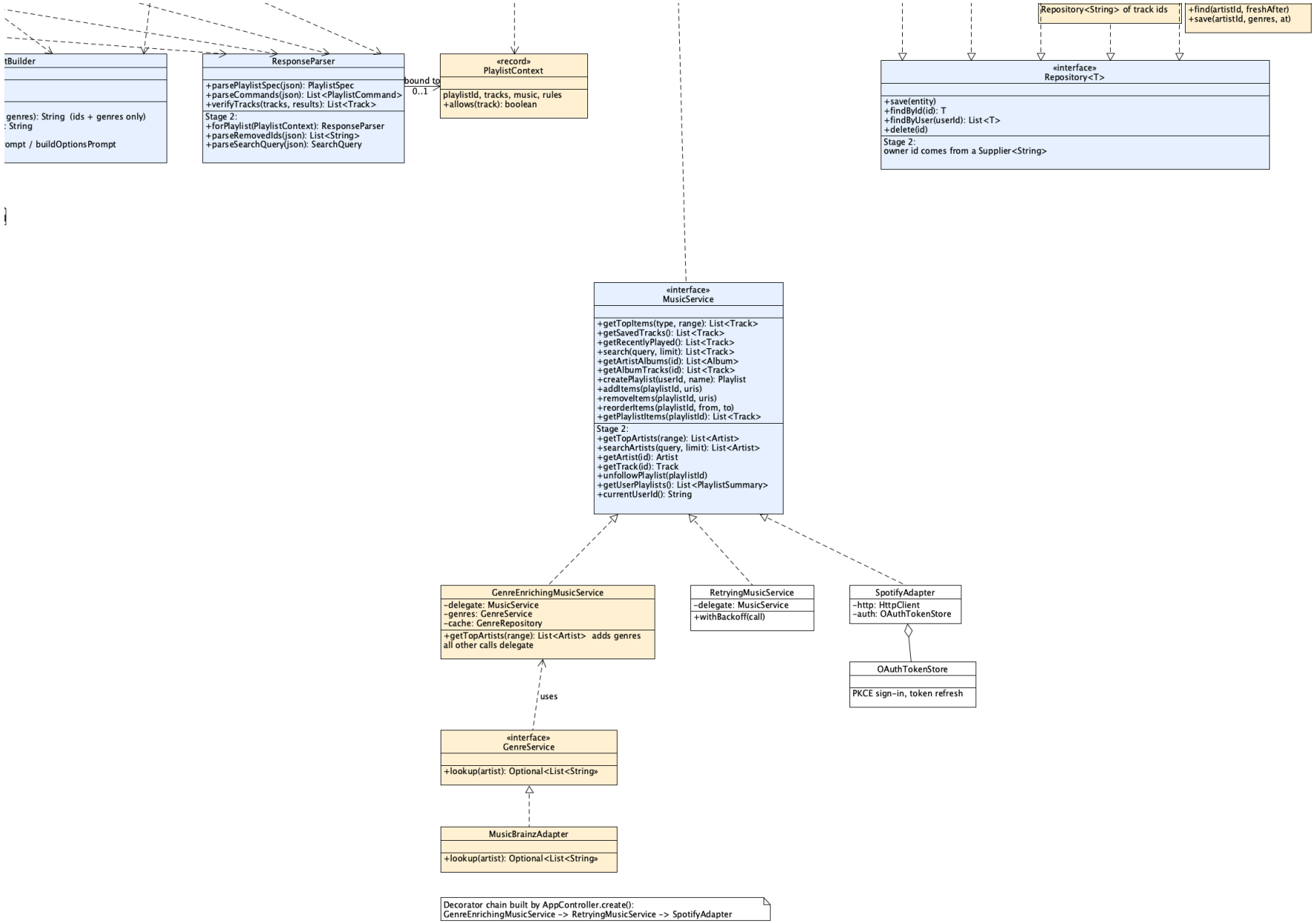


Figure 3.1 (continued). Crops overlap slightly. Colours as in the overview.

3.8 Updated domain model, part 1 of 2 (top half)

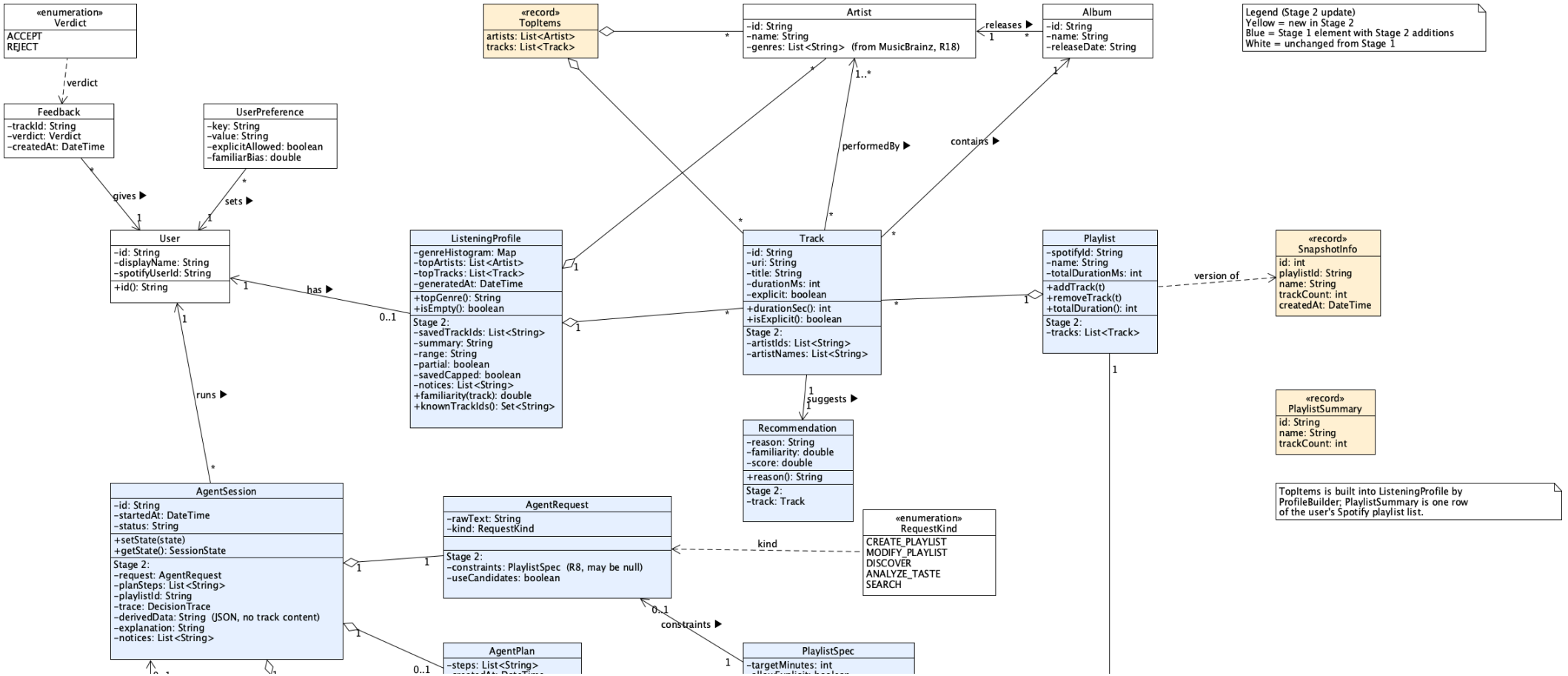


Figure 3.2. Domain model. New types include `TopItems`, `PlaylistSummary`, `SnapshotInfo`, `TrendReport`, `RangeChange`, `SearchQuery`, `FeasibilityReport`, `Relaxation`, `DecisionTrace`; new fields include `PlaylistSpec.blockedTracks`, `AgentRequest.useCandidates`, `Track.artistNames`.

3.8 Updated domain model, part 2 of 2 (bottom half)

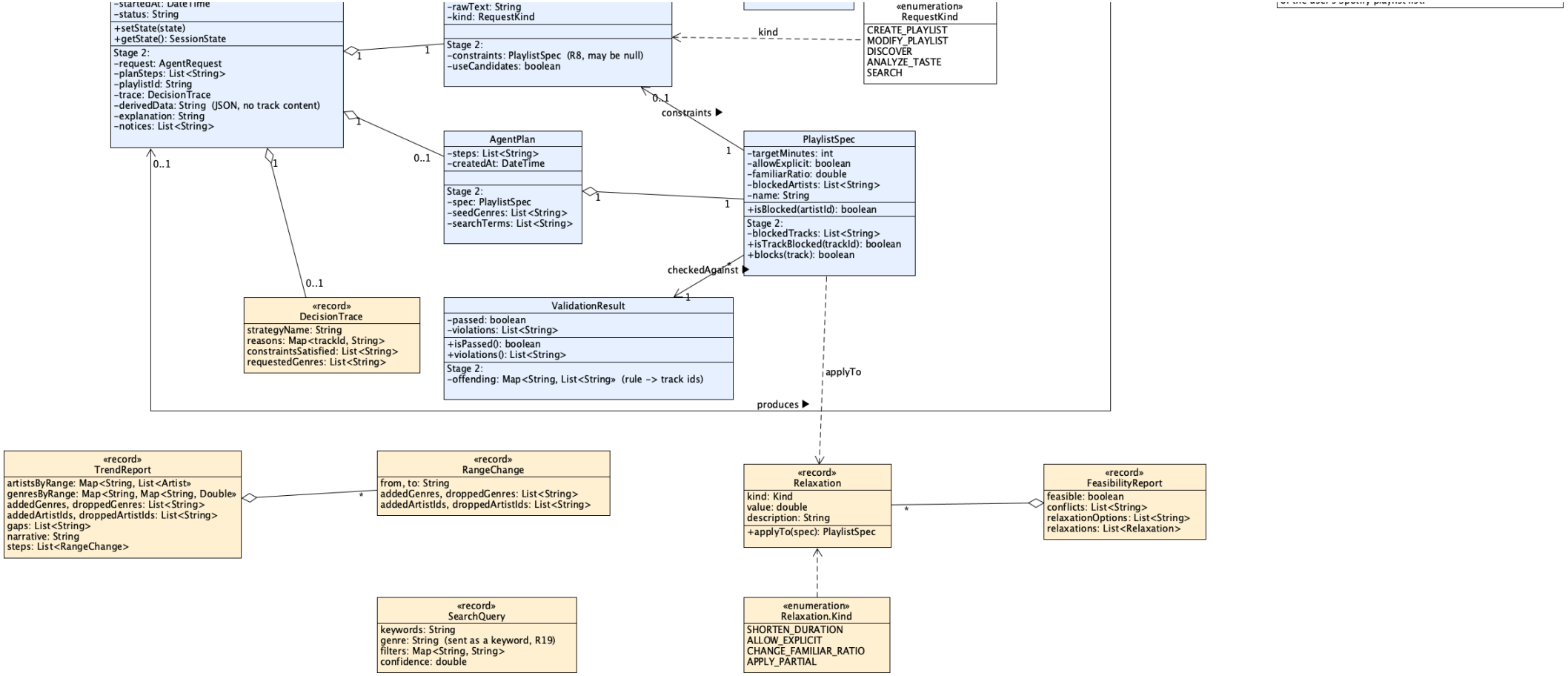


Figure 3.2. Domain model. New types include `TopItems`, `PlaylistSummary`, `SnapshotInfo`, `TrendReport`, `RangeChange`, `SearchQuery`, `FeasibilityReport`, `Relaxation`, `DecisionTrace`; new fields include `PlaylistSpec.blockedTracks`, `AgentRequest.useCandidates`, `Track.artistNames`.

3.9 SD01 create playlist, part 1 of 6 (top left)

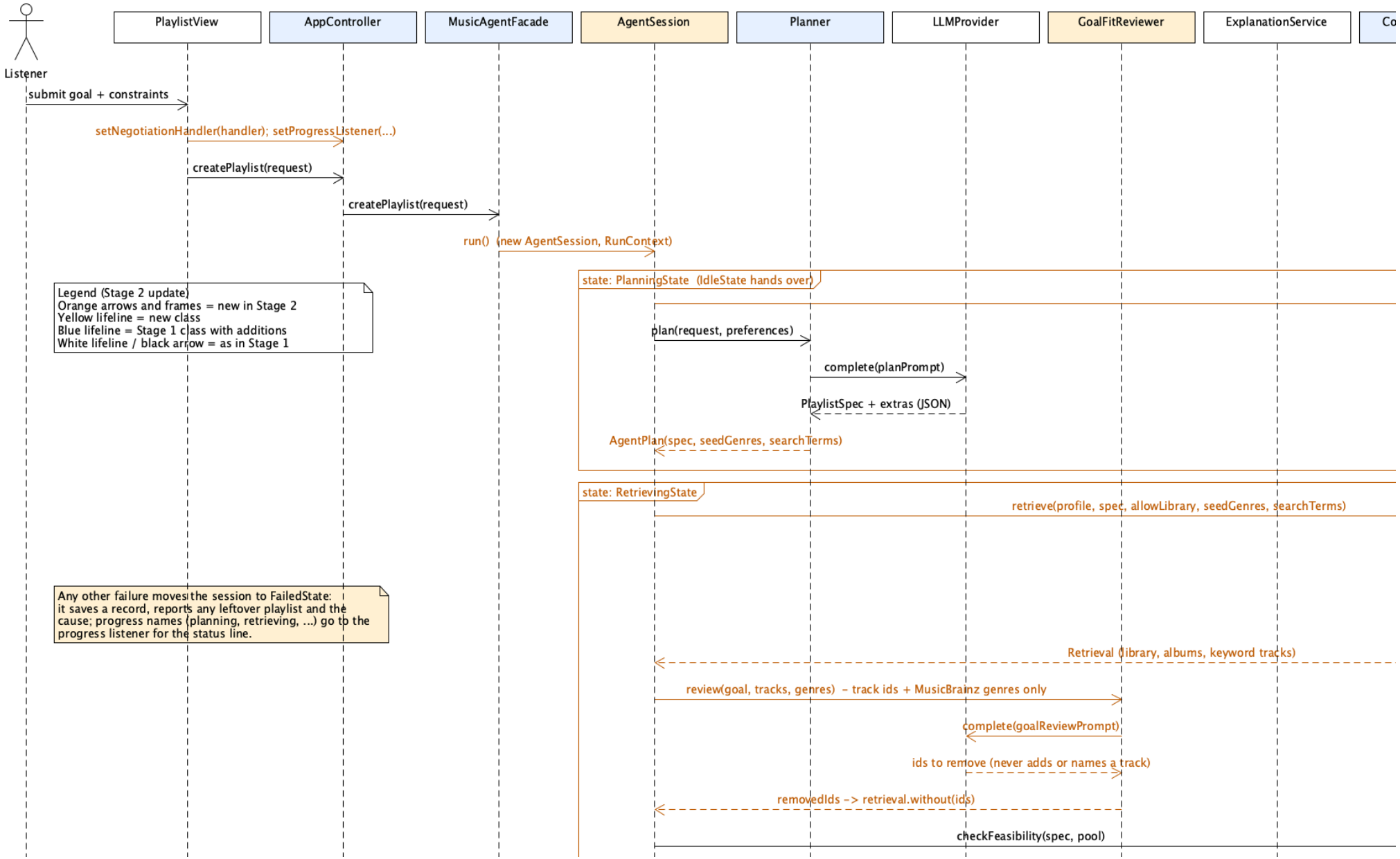


Figure 3.3. SD01 redrawn as the State pattern runs it: Planning, Retrieving (goal review sends only track ids and MusicBrainz genres), Selecting (pick review loop), Validating (loop back), Executing, Done; F11 negotiation and token renewal. Orange = new in Stage 2. Crops overlap slightly; full PNG in `docs/diagrams/png/`.

3.9 SD01 create playlist, part 2 of 6 (top right)

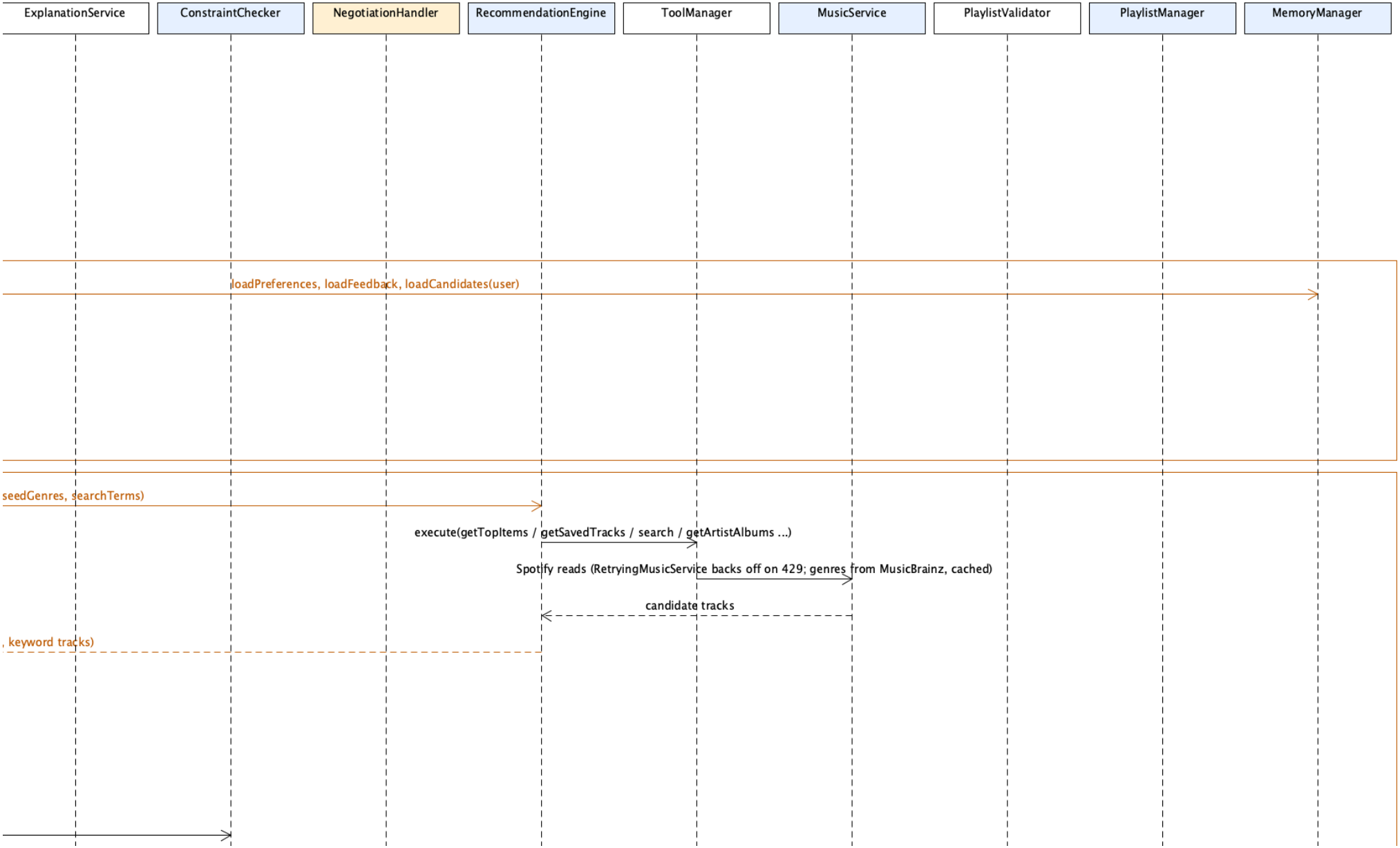


Figure 3.3. SD01 redrawn as the State pattern runs it: Planning, Retrieving (goal review sends only track ids and MusicBrainz genres), Selecting (pick review loop), Validating (loop back), Executing, Done; F11 negotiation and token renewal. Orange = new in Stage 2. Crops overlap slightly; full PNG in `docs/diagrams/png/`.

3.9 SD01 create playlist, part 3 of 6 (middle left)

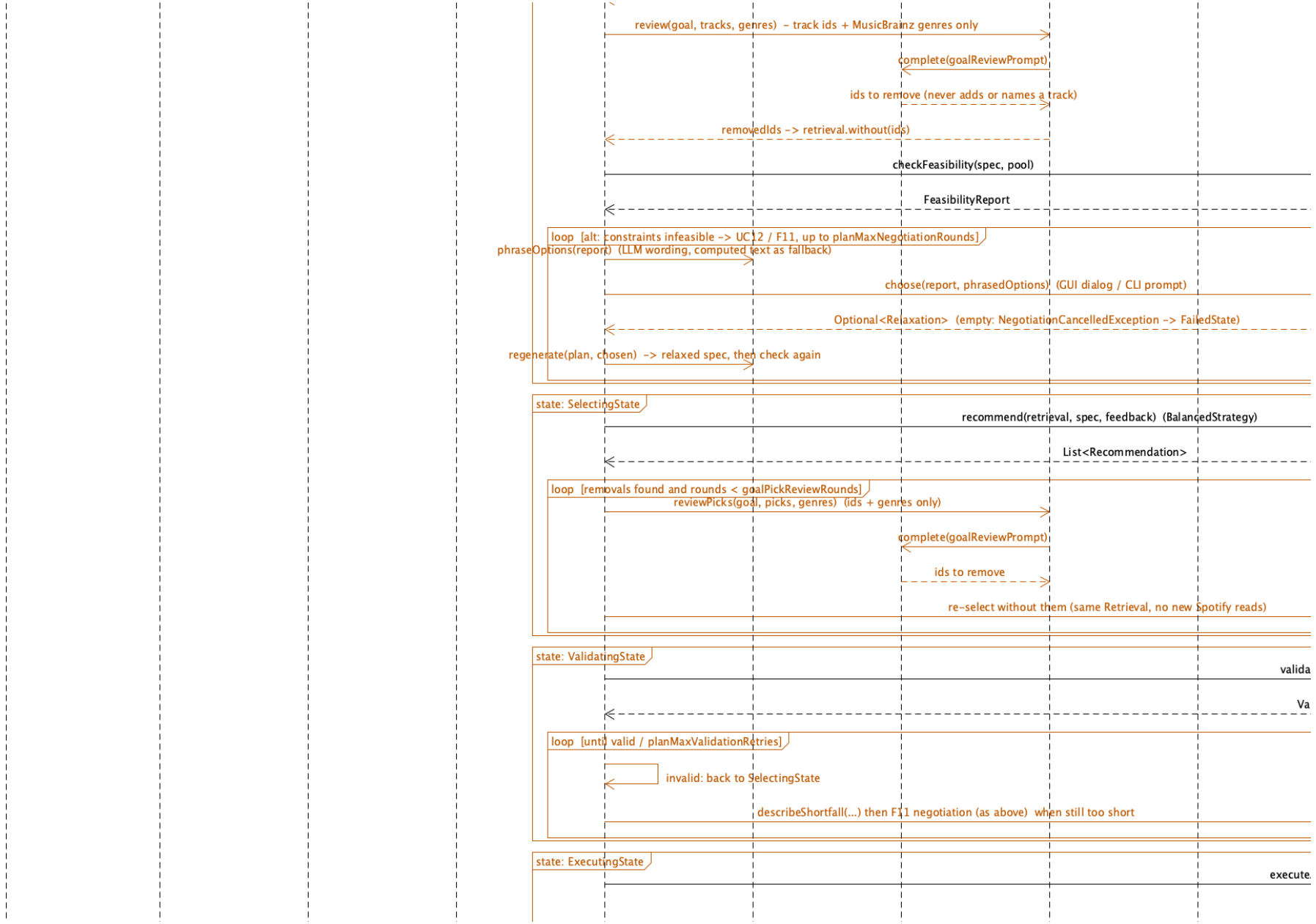


Figure 3.3. SD01 redrawn as the State pattern runs it: Planning, Retrieving (goal review sends only track ids and MusicBrainz genres), Selecting (pick review loop), Validating (loop back), Executing, Done; F11 negotiation and token renewal. Orange = new in Stage 2. Crops overlap slightly; full PNG in `docs/diagrams/png/`.

3.9 SD01 create playlist, part 4 of 6 (middle right)

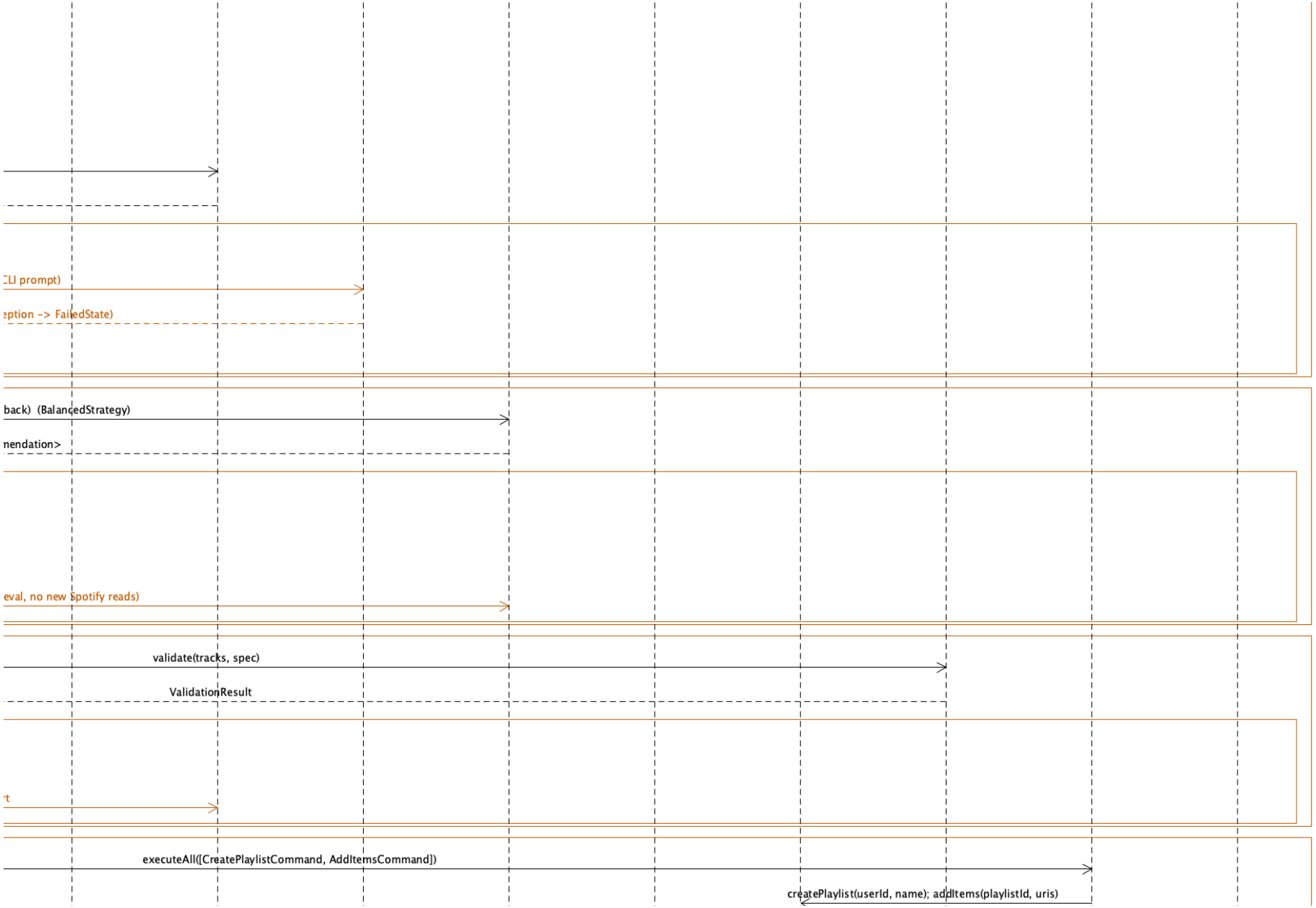


Figure 3.3. SD01 redrawn as the State pattern runs it: Planning, Retrieving (goal review sends only track ids and MusicBrainz genres), Selecting (pick review loop), Validating (loop back), Executing, Done; F11 negotiation and token renewal. Orange = new in Stage 2. Crops overlap slightly; full PNG in `docs/diagrams/png/`.

3.9 SD01 create playlist, part 5 of 6 (bottom left)

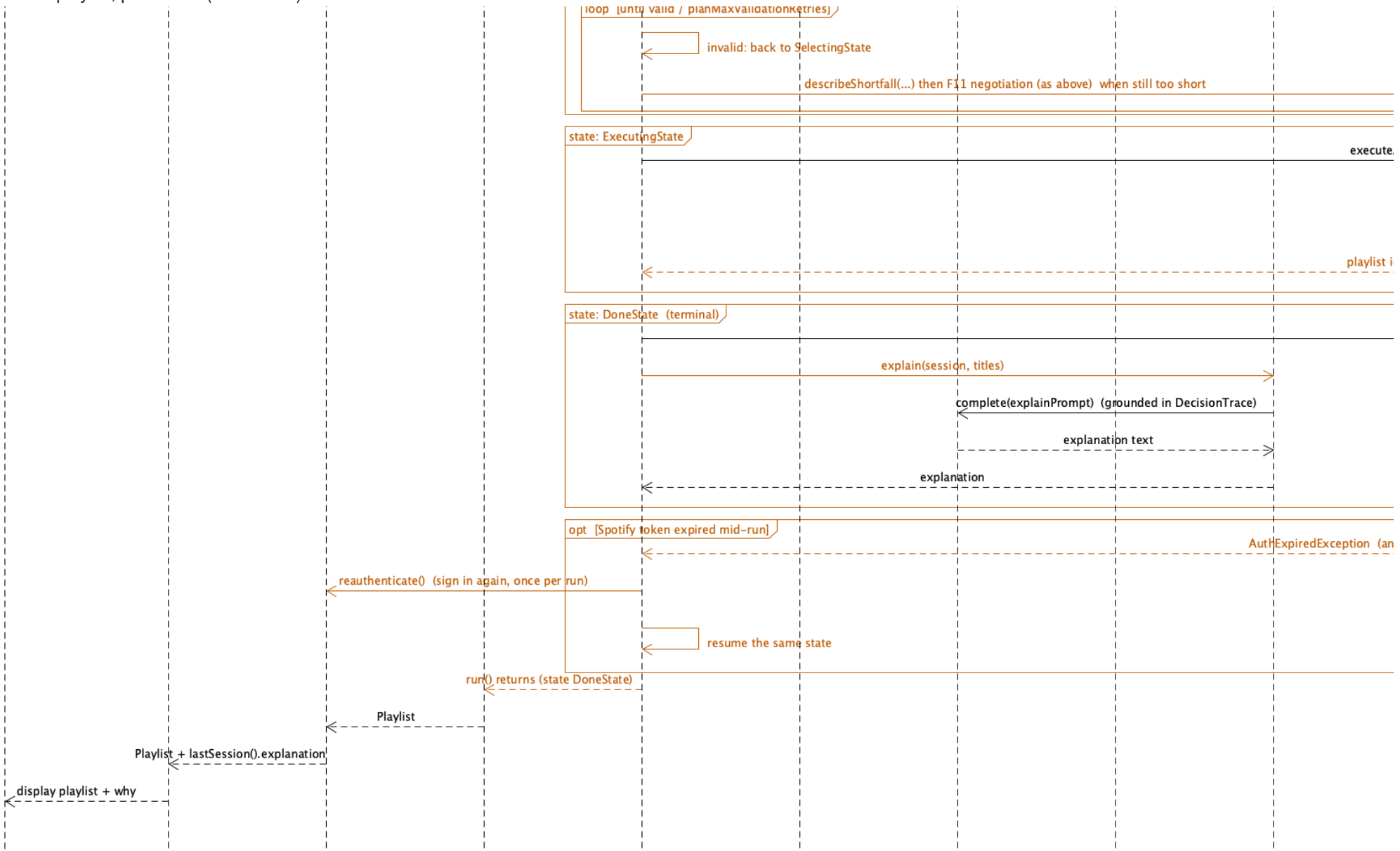


Figure 3.3. SD01 redrawn as the State pattern runs it: Planning, Retrieving (goal review sends only track ids and MusicBrainz genres), Selecting (pick review loop), Validating (loop back), Executing, Done; F11 negotiation and token renewal. Orange = new in Stage 2. Crops overlap slightly; full PNG in `docs/diagrams/png/`.

3.9 SD01 create playlist, part 6 of 6 (bottom right)

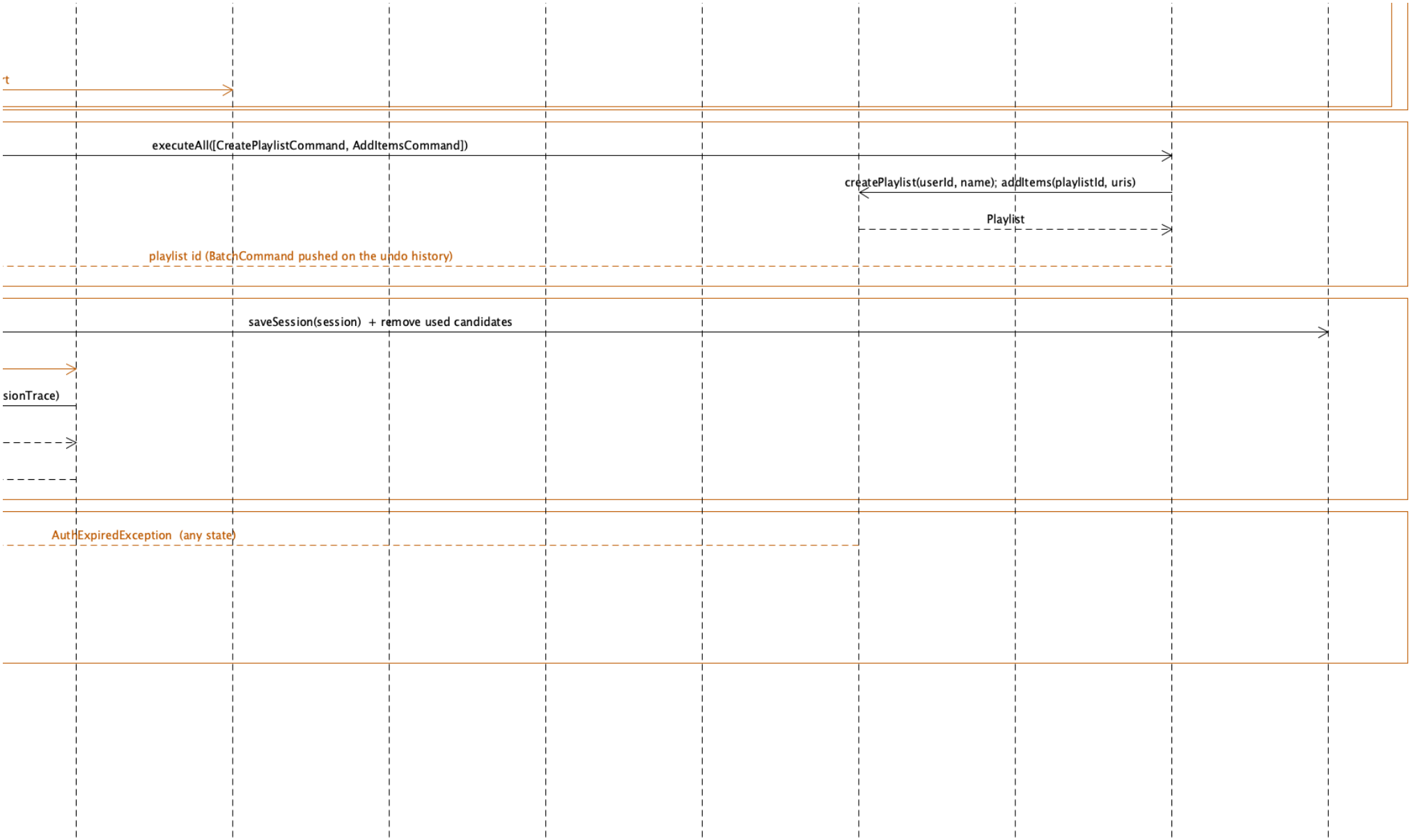


Figure 3.3. SD01 redrawn as the State pattern runs it: Planning, Retrieving (goal review sends only track ids and MusicBrainz genres), Selecting (pick review loop), Validating (loop back), Executing, Done; F11 negotiation and token renewal. Orange = new in Stage 2. Crops overlap slightly; full PNG in `docs/diagrams/png/`.

3.10 SD02 modify playlist with undo, part 1 of 2 (top half)

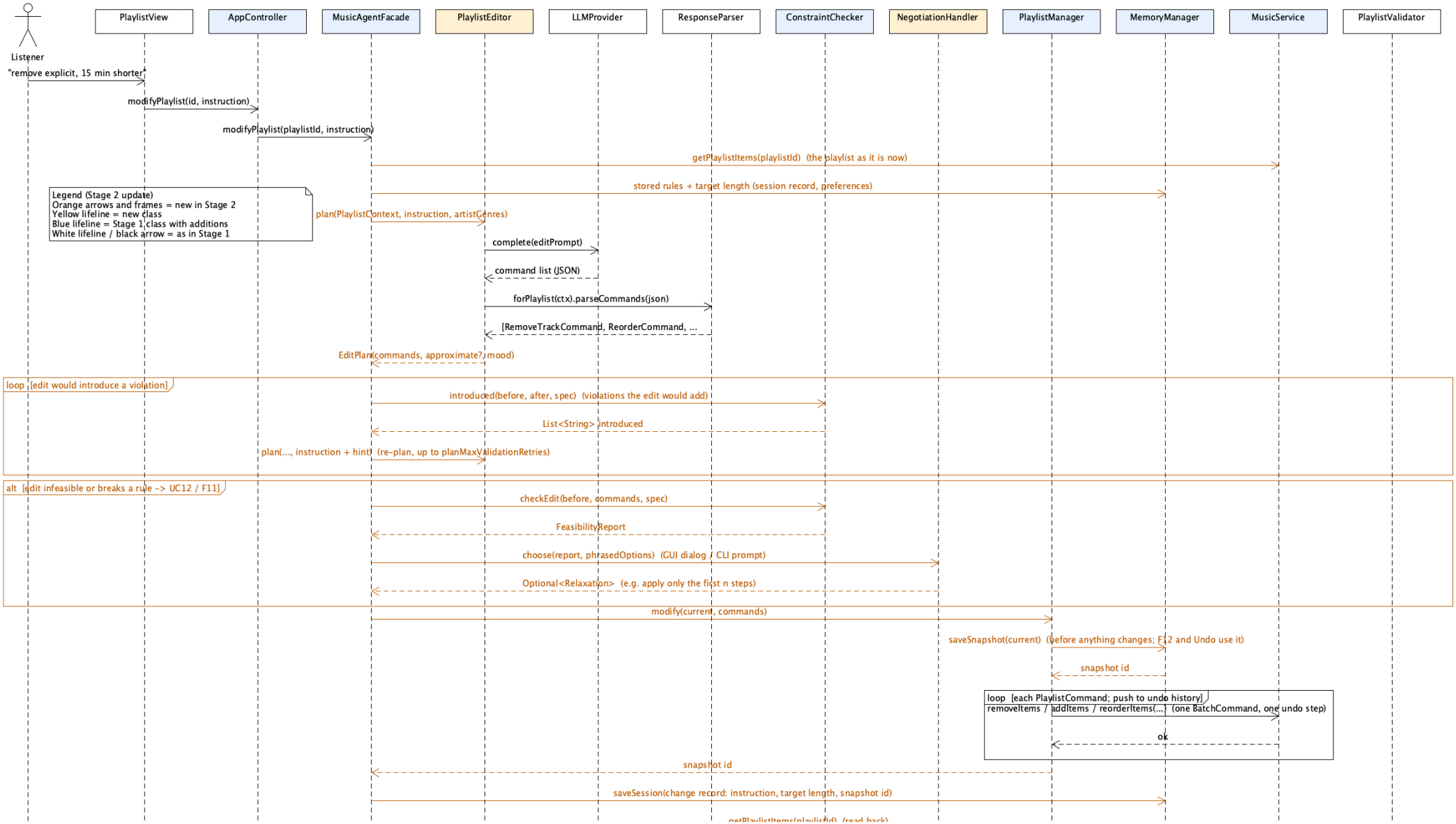


Figure 3.4. SD02: `PlaylistEditor` replaces the direct LLM and parser calls; adjustment loop; F11 partial edit; snapshot before every modify; undo from the latest snapshot when the history is empty.

3.10 SD02 modify playlist with undo, part 2 of 2 (bottom half)

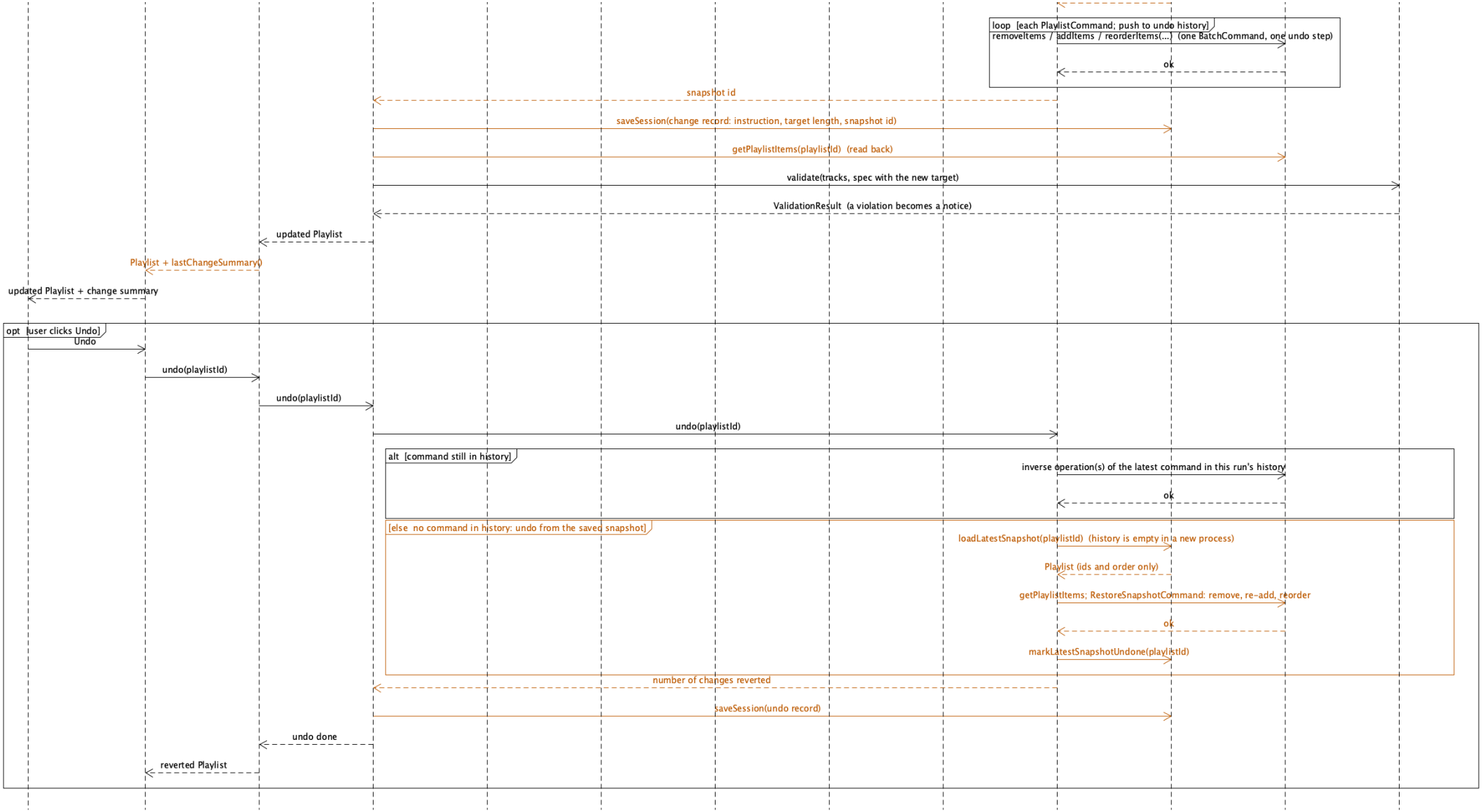


Figure 3.4. SD02: `PlaylistEditor` replaces the direct LLM and parser calls; adjustment loop; F11 partial edit; snapshot before every modify; undo from the latest snapshot when the history is empty.

3.11 SD03 taste profile, part 1 of 2 (top half)

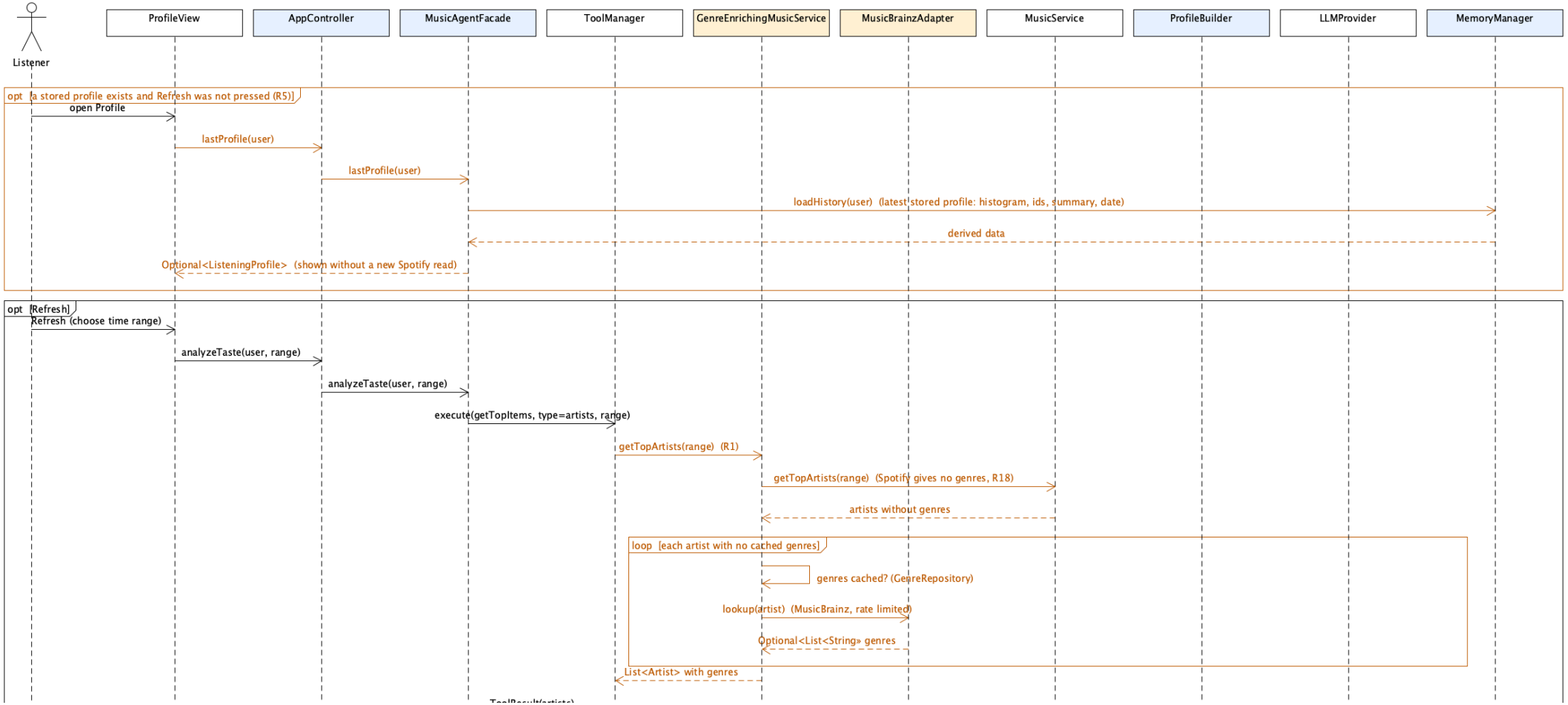


Figure 3.5. SD03: stored profile (R5), genres from MusicBrainz through the decorator and its cache (R1, R18), `TopItems`, labelled fallback summary.

3.11 SD03 taste profile, part 2 of 2 (bottom half)

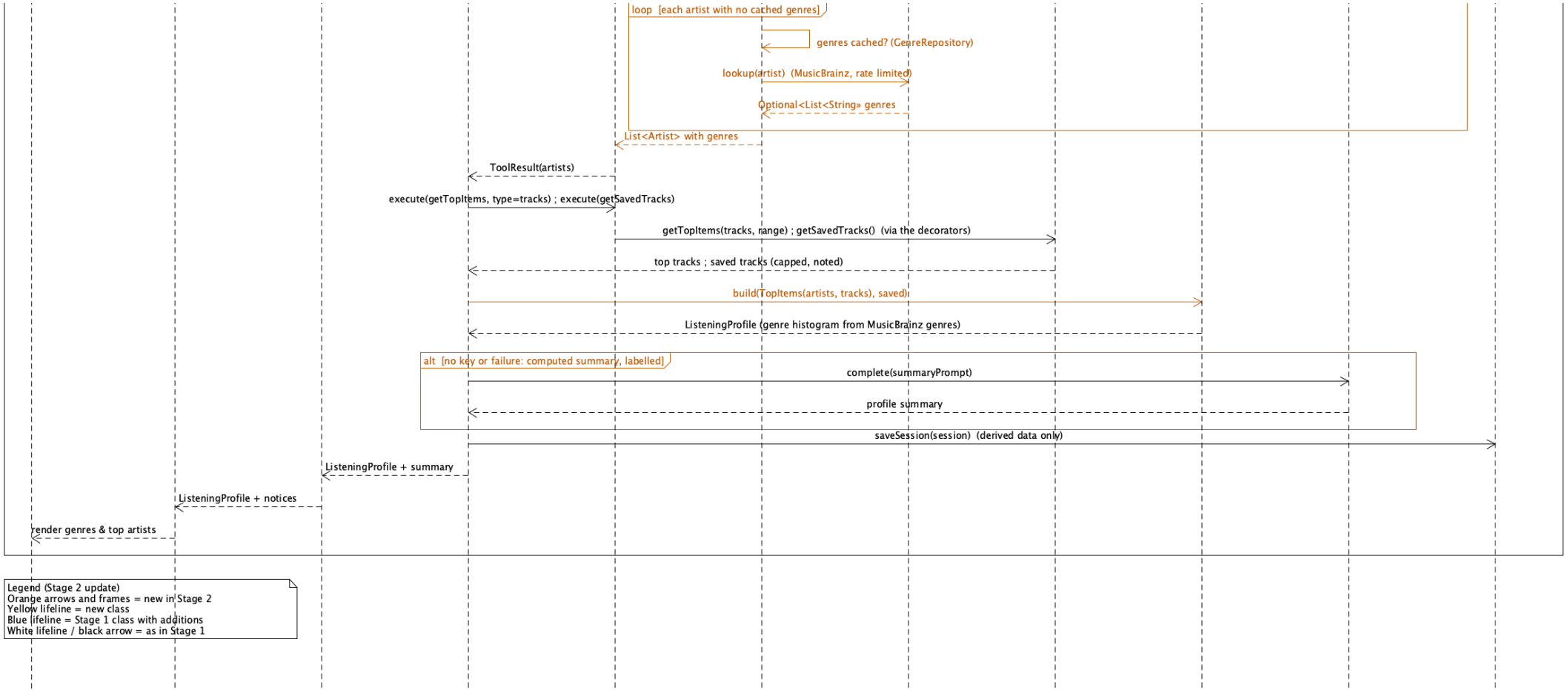


Figure 3.5. SD03: stored profile (R5), genres from MusicBrainz through the decorator and its cache (R1, R18), `TopItems`, labelled fallback summary.

3.12 SD04 discover and feedback

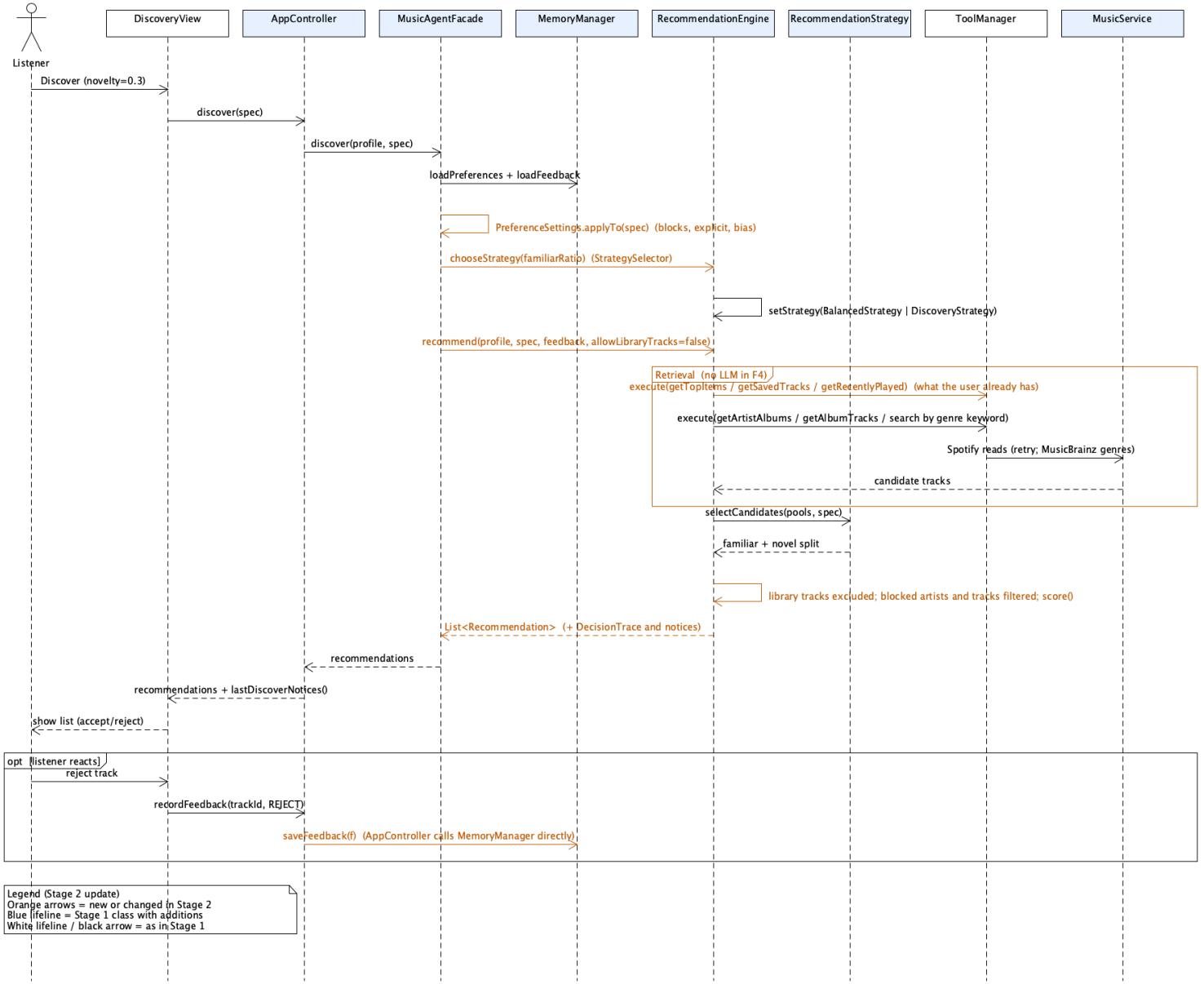


Figure 3.6. SD04: preference merge, strategy choice, library tracks excluded, no LLM call; feedback goes from `AppController` to `MemoryManager`.

3.13 Updated use-case diagram

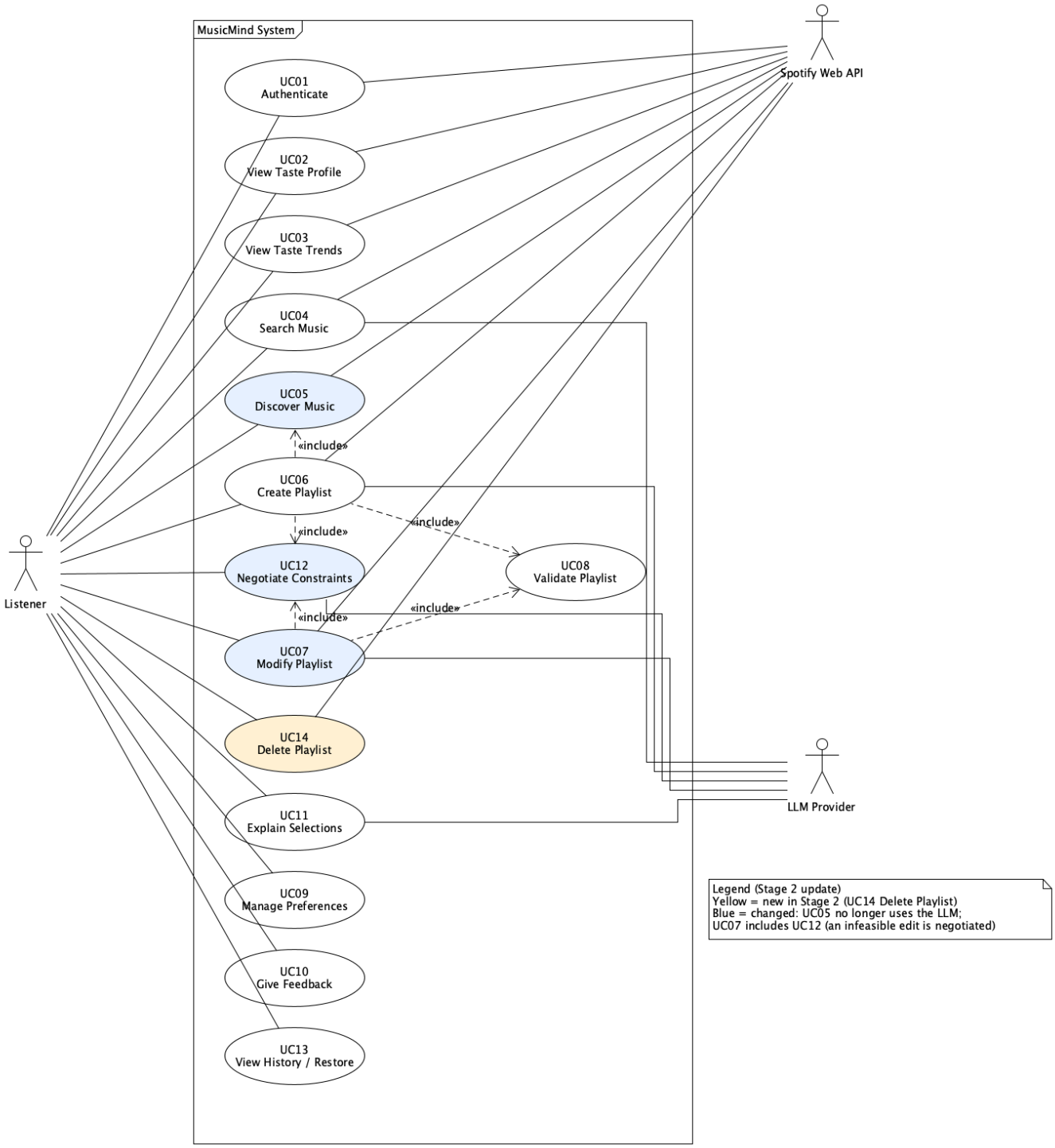


Figure 3.7. Use cases. New: UC14 Delete Playlist. UC07 now includes UC12 (an infeasible edit is negotiated). UC05 no longer uses the LLM (F4 never calls it).

The class and domain diagrams were generated from a script for layout, and every name on them was checked against `src/main` by search. Diagram notes are in `docs/diagrams/README.md`.

4 Changes and rationale

This section is the consolidated record in the form **Original design → Implemented change → Reason → Impact**. 4.1 covers features (items 1-5 of the instructions), 4.2 covers design patterns (items 6-10), and 4.3 covers architecture, class and platform changes. The full entries, with evidence, are in `docs/STAGE2_PLAN.md`. No Stage 1 feature was removed or replaced. One feature is new (Delete). "R1" to "R20" are the rulings in `docs/STAGE2_IMPLEMENTATION_PLAN.md` for places where Stage 1 was silent or inconsistent.

4.1 Feature changes

For each feature: (1) original feature and ID; (2) implemented feature and behaviour; (3) what changed and why; (4) how the original need is still met, or how scope changed; (5) affected use cases, classes, methods and patterns. F9 and F11 were not substantially modified and are noted at the end.

F1 Music Taste Profile (Modified)

1. Stage 1 F1: genres read from Spotify artist metadata; the profile is "persisted".
2. Genres come from MusicBrainz via `GenreEnrichingMusicService`; `profile` shows the stored profile and `--refresh` rebuilds; `--range short|medium|long` added; the stored profile holds ids, counts and the summary only; coverage and a capped library are reported.
3. A live run on October 31 showed Spotify returns no `genres` to this app (R18); Spotify's terms forbid storing content (R4).
4. The listener still gets a genre mix, top artists and a summary; 13 of 20 top artists had genres in the first live run.
5. UC02, SD03; `ProfileBuilder.build(TopItems, saved)` (R2), `MusicService.getTopArtists` (R1), `AppController.lastProfile` (R5); Adapter, Decorator, Facade.

F2 Taste-Change Analysis (Modified)

1. F2: a diff per range, newest against oldest.
2. Every step between neighbouring ranges is compared; genre coverage per range is reported and the wording becomes cautious below 50% coverage.
3. Stage 1 text asks for the diff per range; live data showed many artists have no MusicBrainz genres, so a "dropped genre" was often just missing data.
4. Same need, with honest wording.
5. UC03, SD03; `TrendComparator`, `TrendReport.steps`, `RangeChange`, `PromptBuilder.buildTrendPrompt`.

F3 Natural-Language Music Search (Modified)

1. F3: free text becomes a structured Spotify search; ranked results; "add to candidates".
2. Genre is sent as a plain keyword; mood words are removed from the search text and replaced by a mapped genre; the typed words are searched as well and exact title or artist matches rank first; duplicates are merged; artists named in the query are listed; candidates are stored (ids only) and feed the next `create`.
3. Live: the `genre:` filter returned 0 to 1 tracks where plain keywords returned 20 (R19); mood words matched stock-music titles; the AI's reading drifted from the typed words in the author's GUI test.
4. Same need; search by exact title now works (5 of 5 live runs put exact titles in the top 3). Searching by description is still weak.
5. UC04, SD01; `Planner.parseSearchQuery` (R6), `SearchQuery.toSpotifyQuery`, `SearchRanker.merge`, `CandidateRepository`; Repository, Adapter.

F4 Personalized Discovery (Modified)

1. F4: the LLM is optional and only phrases reasons; exclude tracks the listener knows.
2. Discovery never calls the LLM; reasons are computed from recorded facts; known tracks (top, saved, recently played) are excluded from every pool; widening notices; tracks titled only with a genre word rank lower.
3. A live run found 13 of 20 results came from the user's own history; computed reasons cannot state anything untrue and avoid an extra cost and failure path.
4. The need is met without the optional LLM step, which is a reduction in AI use for this feature.
5. UC05, SD04; `RecommendationEngine.recommend(..., allowLibraryTracks)`, `StrategySelector`; Strategy.

F5 AI Playlist Planning and Generation (Modified)

1. F5 (SD01): plan, retrieve, check feasibility, select deterministically with `BalancedStrategy`, validate, execute, explain. The LLM takes no part in choosing tracks.
2. The same lifecycle through the State classes, plus: new tracks also come from the goal's activity words; a "familiar" track is one by an artist the listener knows; goal scores differ by taste; stored candidates are optional (`--no-candidates`); a failed run rolls the new playlist back; the LLM may *remove* already retrieved tracks that do not suit the goal (the goal-fit review, described below).

3. The author's GUI test on November 2 produced unsuitable songs for a "workout" goal, and Spotify gives this app no tempo or energy data.
4. The need (a playlist that fits the goal) is addressed more directly, but the change relaxes the Stage 1 and plan statement that the LLM takes no part in selection, and its effect on playlist quality is not yet proven.
5. UC06, UC12, SD01; `GoalFitReviewer`, `RetrievingState`, `SelectingState`, `Retrieval`, `CandidateRetriever.byKeywords`, `RecommendationEngine.score`; State, Strategy, Command.

F6 Natural-Language Playlist Modification (Modified)
1. F6 (SD02): the LLM parses the instruction, `executeAll` applies commands, then validate; Undo calls `PlaylistManager.undo`.
2. A `PlaylistEditor` turns the instruction into commands (mood-only edits use a labelled heuristic); before any write the edit is simulated and re-planned if it would add violations; a snapshot is saved first; undo works across CLI runs from snapshots; the playlist's target length moves with a stated length change; `playlists` and `show` list and open playlists.
3. Each CLI run is a new process, so an in-memory undo stack alone cannot undo (R10); Stage 1 said "opening a playlist" but no call listed playlists; the old target is wrong after a deliberate length change.
4. Same need, with more safety.
5. UC07, UC12, SD02; `PlaylistEditor`, `PlaylistManager.modify/undo`, `ConstraintChecker.checkEdit`, `PlaylistTargets`; Command.

F7 Playlist Constraint Validation (Modified)
1. F7: explicit, duplicate, duration, blocked-artist and min/max rules, judged against the spec saved at creation.
2. Blocked *tracks* added (Stage 1 lists "artists/tracks" but only artists existed); the blocks in force now judge every playlist; the newest recorded target length is used; `offending()` lists track ids.
3. A block stands for the listener's current wishes, so removing or adding one must affect existing playlists.
4. Same need, with one deliberate difference from "validate against the saved spec" for blocks only.
5. UC08; `PlaylistValidator`, `PlaylistSpec.blockedTracks`, `PreferenceSettings.withCurrentBlocks`.

F8 Preference Memory (Modified)
1. F8: preferences including blocked artists, stored in PostgreSQL.
2. Artists are typed by name and stored as Spotify ids; `findArtists`, `findTracks`, `blockArtist(id)` and `blockTrack(id)` let a caller block what the user chose; blocking is refused when the stored preferences cannot be read.
3. Users know names, not ids; a typo or a shared name blocked the wrong artist; an outage must not erase blocks. Names are read for display and never stored (Spotify terms).
4. Same need.
5. UC09; `PreferenceSettings`, `AppController.findArtists/blockArtist`, `MemoryManager`; Repository.

F10 Grounded Recommendation Explanation (Modified)
1. F10: the LLM explanation may use only recorded facts.
2. `verifyExplanation` also rejects unrecorded numbers, quoted names, genres and preference claims; the trace records the requested genres; titles are shown from the run (display only, never stored); the prompt forbids saying a track fits a mood unless its own reason says so; `explain` accepts a session id or a playlist id.
3. A live explanation said "genres you love" for genres that came from the goal; users know playlist ids.
4. Same need, stricter.
5. UC11; `ExplanationService`, `DecisionTrace.requestedGenres`, `ResponseParser.verifyExplanation`, `MemoryManager.findSessionByPlaylist` (R13).

F12 Agent History and Snapshot/Restore (Modified)
1. F12: `getHistory` and `restore` replay a snapshot through `PlaylistManager`; opening a past session shows its plan and result.
2. `history <id>` opens one session; a restore saves the current content first so it can be undone; a missing playlist is recreated from a snapshot; snapshots hold ids and the name only (R14); a restore of an unchanged playlist says so.
3. Spotify's terms forbid storing content; `history` originally printed one line per session; recreating covers delete.
4. Same need.
5. UC13, SD02; `MemoryManager.loadSnapshot`, `PlaylistManager.restoreFrom`, `RestoreSnapshotCommand`, `AppController.recreateFromSnapshot`; Command, Repository.

New: Delete a playlist (Added)
1. New; Stage 1 has no delete.
2. `delete <playlistId>` (CLI) unfollows a playlist the listener owns after saving a snapshot and asking for the name to be typed.
3. In user testing a playlist could not be removed from MusicMind (Spotify's API has no true delete, only unfollow).

4. Scope grows by one feature; the way back is F12.
5. UC14 (new); `DeletePlaylistCommand`, `PlaylistManager.delete`, `MusicAgentFacade.deletePlaylist`, `MusicService.unfollowPlaylist` (R11, with the extra OAuth scope `user-library-modify`); Command. No GUI control yet.

F9 and F11 (not substantially modified)
F9: the only difference is that the track id is validated (22 letters or digits, or a `spotify:track:` URI) before saving, because a live test saved the literal text `TRACK_ID`. F11: the only differences are extra relaxation kinds (shorten, change the share, allow explicit, apply only part of an edit) and that negotiation also runs during modify, as Stage 1 UC12 says (at first only create negotiated).

The LLM's role: the goal-fit review and the November 3 policy correction

Original. Stage 1 gives the LLM no part in choosing tracks, and the implementation plan's constraint 2 says "the LLM never selects tracks and never receives bulk catalogue data".

Change. Since November 2 the LLM reviews real tracks that were already retrieved and may only return ids to *remove* (`GoalFitReviewer`, in `RetrievingState` and again on the final picks in `SelectingState`, bounded per run). It never names or adds a track, ids it was not given are ignored, and selection and order stay deterministic. The review is best effort: with no LLM, a failure, a malformed reply, or a reply that would remove too much, nothing is removed and a notice says it was skipped.

Reason. The author's live test produced unsuitable "workout" songs and Spotify gives no audio features.

Correction on November 3. The first version sent OpenAI each track's id, title and artist names. Spotify Developer Policy section III item 14 forbids ingesting Spotify Content into an AI model. This was found while writing the documentation and fixed the same day (commit 1b18d24), test first: the prompt now carries only the track id and the genres MusicMind already knows for it (from the search that found it, the listener's profile, or cached MusicBrainz genres). A track with no known genre is not sent and is kept. A reflective test (`PromptBuilderTest`, hardened in commit 4211b8b) calls every `PromptBuilder.build*` method and fails if a prompt contains a track title or artist name.

Impact. The constraint now holds in this form: ids and MusicBrainz genres, no Spotify titles or names. The review is less precise than a title-based one and cannot judge tracks without a genre. It is not proven to improve playlists on the real account, and it costs up to three extra LLM calls per create.

Open question. The F6 edit prompt sends each track's id, explicit flag and length in minutes (no titles or names). Whether that counts as Spotify Content is undecided and is for the author to decide.

4.2 Design pattern changes

Items: (6) original pattern; (7) pattern implemented; (8) participating classes and roles; (9) design problem; (10) why appropriate and effect on the architecture. **No pattern was replaced or dropped.** The five counted patterns are the Stage 1 patterns; the changes are additions inside them, plus one new Adapter and one new Decorator. Participant tables are in Section 3.

Pattern	6-7 Original and implemented	8 New or changed participants and roles	9-10 Problem, why, effect
Adapter	Stage 1: `MusicService/SpotifyAdapter`, and `LLMProvider`. Implemented: the same, plus a second adapter pair.	`GenreService` (target) and `MusicBrainzAdapter` (adapter) supply genres; `OpenAiProvider` is the only LLM adapter built by the app.	Spotify sends no genres to this app (R18). A second source behind an interface keeps the core independent of it. Effect: F1, F2, F4 and F5 get a genre signal again.
Strategy	Same pattern. Added `bindProfile` (R7) and `StrategySelector`.	`StrategySelector` picks one of the three strategies; `bindProfile` gives helpers the profile.	Stage 1 helpers needed a profile its signature did not carry. Effect: the Stage 1 signature is kept; an unbound strategy throws a clear error.

Pattern	6-7 Original and implemented	8 New or changed participants and roles	9-10 Problem, why, effect
Command	Same pattern, five commands. Added three commands and undo from snapshots.	`BatchCommand` (one undo step per edit), `RestoreSnapshotCommand` (F12), `DeletePlaylistCommand` (new feature); `PlaylistManager` snapshots before every change.	CLI runs are separate processes (R10), restore and delete need the same reversible machinery. Effect: undo, restore and delete share one mechanism; failures roll back.
State	Same eight states. Added support classes.	`RunContext` (data of one run), `RunServices` (collaborators), `Negotiator` (one F11 round).	Keeps state classes small and the phase logic in the states. Effect: no change to the pattern, easier tests.
Facade	Same pattern.	`AppController` gained `last*()` accessors, `login`, `setNegotiationHandler`, `setProgressListener` (R9, R16).	Both interfaces need the finished session, the F11 choice and progress without putting interface code in the core.
Decorator (not counted)	Stage 1: `RetryingMusicService`. Implemented: plus `GenreEnrichingMusicService`.	The new decorator wraps `MusicService` and fills genres through `GenreService`, with a cache.	Genre lookup is added without changing `SpotifyAdapter`.
Repository (not counted)	Stage 1 repositories. Plus two new.	`CandidateRepository` (F3 candidates) and `GenreRepository` (genre cache).	Persistence details stay out of the agent.

4.3 Architecture, class and platform changes

Original design	Implemented change	Reason	Impact
PostgreSQL named, no setup detail	`memory/Database` creates six tables on start; tests use embedded PostgreSQL; the app runs with safe defaults and a warning when the database is down.	A reviewer must build with nothing installed; Stage 1 asks for the database error case.	README explains the one-time role and database creation.
Gradle 8.14.3; JavaFX 17.0.6	Gradle 9.1.0 with a toolchain resolver; JavaFX 21.0.13; `bin/musicmind` and `bin/musicmind-gui` from `installDist` (R15).	Gradle 8 cannot run on JDK 25; JavaFX 17.0.6 crashed at launch on macOS 26.	Builds on JDK 17 to 25; the project still compiles as Java 17.
No configuration named	Every tunable and every user-facing sentence is a key in `musicmind.properties`, read through `Config`; secrets from `.env`.	Stage 2 rejects hard-coded demonstration content.	No key, id or canned result in `src/main`.
A mock LLM implied for tests and demos	`OpenAiProvider` is the only provider the app builds; `MockLLMProvider` is test-only (R17).	Canned answers are not functionality.	With no key, features say so or use labelled fallbacks.
Spotify endpoints as in Stage 1	February 2026 development-mode shapes: reads `item`, searches at most 10 per call and pages, never calls `/recommendations`, audio features, related artists, popularity or batch artist reads.	Those calls are closed to this app.	Discovery is built from the listener's own data, as Stage 1 already says.
Genres filtered with `genre:`	Genre sent as a plain keyword (R19).	The filter returned 0 to 1 tracks in live tests.	F3 and F4 return results.
Signatures unspecified in places	Additive members only: R1 to R16 (for example `getTopArtists`, `TopItems`, user-aware repositories, `AgentRequest.constraints`, `unfollowPlaylist`, `login`). No Stage 1 name or signature changed.	Stage 1 was silent or inconsistent.	Stage 1 class and member names all still exist (checked by search over `src/main`).

Original design	Implemented change	Reason	Impact
Classes in the Stage 1 diagram	R20: small supporting classes (for example `Retrieval`, `CandidateRetriever`, `GoalFitReviewer`, `PlaylistEditor`, `RunContext`, `GuiContext`, `ViewFormat`); the full list is in `docs/STAGE1_CONFORMANCE.md`, section 6.	Keep each Stage 1 class focused.	Shown on the updated class diagram.
GUI as listed in Stage 1	Seven plain JavaFX tabs, a Create dialog and an F11 dialog; one background worker; friendly failure wording. No Stage 1 GUI interaction was dropped. A redesign is planned and not started.	`AppController` keeps mutable `last*()` state, so calls must not overlap.	The videos must show the GUI as it is.

5 AI tools and collaboration log

Using AI development tools was mandatory in Stage 2. The AI wrote code and drafts; the author set the rules, chose directions, ran the checks that need her own Spotify and OpenAI accounts, reviewed the changes and decides what is merged. Every task was built test first and then reviewed by a separate agent, and the larger pull requests were reviewed again before merging. The full log is `docs/AI_COLLABORATION_LOG.md`; it holds no keys.

5.1 Tools and models

Tool / platform	Model	Used for	Total tokens	Money cost
Claude Code (Anthropic's coding agent, desktop app and CLI), as the coordinating session	Claude Opus 5.5	Planning, the rulings R1-R20, task briefs, code review of each task and pull request, live checks and fixes, the Stage 2 documents.	**[to be filled by Sarina]**	**[to be filled by Sarina]**
Claude Code subagents	Claude Sonnet 5.5	Implementing each task test first (`ProfileBuilder`, `TrendComparator`, `RecommendationEngine`, the create flow and state classes, the Command classes and `PlaylistManager`, the CLI, the GUI views, the backend fixes, the documentation, this report) and independent review of each task.	**[to be filled by Sarina]**	**[to be filled by Sarina]**
OpenAI API (the app's own `LLMProvider`)	`gpt-4o`	Not a development tool: it runs inside MusicMind (goal and search parsing, summaries, option wording, explanations, goal-fit review).	**[to be filled by Sarina]**	**[to be filled by Sarina]** of the USD 10 credit paid

Cost cells are blank on purpose: the numbers cannot be read from the repository. Where to read them: the Claude usage or billing page for Claude Code, and platform.openai.com (usage page) for the OpenAI API. Claude Code runs under the author's own subscription. The Spotify Web API (development app) and MusicBrainz are free. **[Sarina: add a row here for any other AI tool used in Stage 2, with its model and purpose.]**

5.2 Collaboration log (by session)

Date	Task	AI contribution	Human (Sarina) contribution	Outcome
Oct 30	Scaffold, F1 spine	Gradle and JavaFX build, domain model, interfaces, `PlaylistValidator`, strategies, `SpotifyAdapter` (PKCE), `RetryingMusicService`, `AppController`, `MusicAgentFacade`; F1 in CLI and GUI.	Reviewed the design, confirmed the stack, approved installs.	Compiling skeleton and F1; not yet live.
Oct 30	Portable build; Tasks 1-5 foundation	Gradle 9.1 and JavaFX 21 fixes; PostgreSQL, Feb 2026 Spotify shapes, `OpenAiProvider`, `Config`, `ToolManager`, `PromptBuilder`, `ResponseParser`.	Approved the plan and the rulings R1-R20, including no hard-coded results in `src/main`.	Build green with no keys; nothing live yet.

Date	Task	AI contribution	Human (Sarina) contribution	Outcome
Oct 31	Tasks 6-8: F1-F4, F8, F9	`ProfileBuilder`, `TrendComparator`, `SearchRanker`, `RecommendationEngine`, `StrategySelector`, CLI commands. Reviews found and fixed an LLM failure that could skip the fallback and widening that overwrote the chosen strategy.	Ran the live checks with real keys.	285 tests; then live.
Oct 31	Live test, genre fix	Diagnosed "(no genre data)", tested MusicBrainz, built `GenreEnrichingMusicService`, `MusicBrainzAdapter` and a cache; review caught a missing timeout.	Ran the first live test, found the gap, chose MusicBrainz.	Genres for 13 of 20 top artists.
Nov 1	Tasks 10-12: F5, F11, F6, F7, F10, F12, CLI	Planner, `ConstraintChecker`, eight session states, `PlaylistEditor`, explanation checks, facade and controller methods, the complete testable CLI, install scripts.	Reviewed diffs; ran `create`, `modify`, `undo` and others live.	Live: create, validate, explain, history, modify, undo.
Nov 1	PR #1 review and fixes; live undo check	Whole-PR review; reproduced an undo defect with a test; fixed 10 findings test first (one-use snapshots, real Spotify positions, no blind retry of unsafe writes); recovery when a move fails part way.	Asked for the review before merging; approved the live check on a test playlist.	All findings fixed; tests pass.
Nov 1	Progress report, Stage 1 gaps	Found six gaps against Stage 1 and closed them test first (playlist list, session detail, all trend steps, blocked tracks, block by name, candidates).	Read the report and chose to close the gaps before the GUI.	Checked live, including a create using a candidate.
Nov 2	Task 13: GUI	Seven JavaFX tabs, Create and F11 dialogs, one worker over `AppController`. A reviewer found garbled thumbs labels, a silent bad-length entry and a swallowed expired login; the PR code review found the Create button could never start.	Set the rule of no Spotify writes without her say-so; ran the write checks.	GUI PR #4 open; read-only live run passed.
Nov 2	Backend fixes from the GUI test; final review round	Exact-title search, duplicates merged, optional candidates, activity searches and goal review, scoring floor, block by choice, delete; review findings (120-track review, pick review, stock-title penalty).	Ran the GUI test that produced the findings; set the rulings.	842 tests; goal fit not re-run live.
Nov 3	Policy fix: goal review sends no Spotify titles	Found the title and artist names in the review prompt while writing the documentation; fixed test first.	Decided the fix and reviews it.	Review sends ids and genres only.
Nov 3	Task 14: documentation; updated UML; this report	Rewrote the plan and conformance documents, README, updated the UML sources and PNGs, wrote this report and its build script.	Task rules: say only what was tested live; supplies costs and video links.	Documents match the code as of this commit.

5.3 Representative interactions

1. **A live result changed the design (Oct 31).** The author ran `profile` on her real account and saw "(no genre data)". Claude Code did not guess: it checked the raw Spotify reply, found no `genres` field for this app, tested MusicBrainz, and built a Decorator over `MusicService`. The author chose MusicBrainz as the source. Reviewers then caught a missing timeout and a crash on a malformed reply, both fixed (R18).
2. **A review found what the tests missed (Nov 2).** After 768 tests and the per-task reviews had passed, the PR code review found that the GUI Create button could never start, because it disabled a slider that was bound to a property. It was fixed and a write-free script confirmed the click path.
3. **A user test led to a change from Stage 1 (Nov 2).** The author's GUI test produced a "workout" playlist with unsuitable songs. Because Spotify gives this app no audio features, the plan was to let the LLM review retrieved tracks and remove those that do not fit, with the rule that it may only return ids to remove. This is recorded as a difference from Stage 1.
4. **The AI found its own policy error (Nov 3).** While documenting the goal review, Claude Code noticed the prompt carried titles and artist names, checked this against Spotify Developer Policy section III item 14, and reported it. The author decided the fix, and a guard test now prevents a repeat.

6 Development and verification summary

6.1 Weekly commit summary

Dates are Toronto time. A "code commit" here means a commit that changes files under `src/` or the build files (`build.gradle`, `settings.gradle`), excluding merge commits; counts come from `git log` with `TZ=America/Toronto` on November 3. The instructions also exclude empty, report-only and formatting-only commits. The counts below are by file path, so they include a few small commits (for example a tidy of test comments) and are an upper bound; the lists below show substantive ones. Commit dates are authoring dates and links open once the branch holding the commit has been pushed (the documentation branch has not been pushed yet).

Period	Dates (inclusive)	Required	Code commits (non-merge)	Status
1	Oct 20-26	5	0	**Missed.** There were no commits at all in this period: Stage 1 was committed on October 9 and the first Stage 2 commit is on October 30. The single 5-point deduction applies.
2	Oct 27-Nov 2	5	147 (Oct 30: 23, Oct 31: 28, Nov 1: 55, Nov 2: 41)	Met. There were no commits on Oct 27-29.
3	Nov 3-9	5	2 so far (Nov 3; 3 by path)	**In progress.** The minimum of 5 is not yet met; this table counts only commits that exist today. By path, a third commit on Nov 3 touches `src/`, but it only edits `src/README.md` and is documentation. Before this report was added, the other commits of Nov 3 (up to `6803a41`, 17 documentation, diagram and housekeeping commits and one merge of the policy fix) do not count toward the minimum; the report and its fix commits are documentation too.

Period 1 (October 20-26)

No commits. This is a real miss and is not hidden: the work started on October 30.

Period 2 (October 27 - November 2): 147 code commits; a representative selection

Commit	Toronto time	What it did
`3d6644e`	Oct 30 14:18	Gradle and JavaFX build setup and wrapper
`cb0c8ee`	Oct 30 14:18	Domain model (Track, Playlist, PlaylistSpec, AgentSession, ...)
`e1bb45f`	Oct 30 14:18	Core interfaces: MusicService, Tool, LLMProvider, RecommendationStrategy, PlaylistCommand, Repository, SessionState
`f8f6283`	Oct 30 14:20	Deterministic PlaylistValidator (F7) with unit tests
`515e8d9`	Oct 30 14:20	Strategy pattern: the abstract, Familiar, Discovery and Balanced strategies
`a6d6570`	Oct 30 14:56	SpotifyAdapter (Adapter) with PKCE sign-in
`e694438`	Oct 30 14:57	RetryingMusicService decorator for 429 backoff, with tests
`d784da6`	Oct 30 20:12	Replace the first SQLite storage with PostgreSQL persistence and the Stage 1 repositories
`2a1f0d0`	Oct 30 20:24	OpenAiProvider (replacing the first provider) and central tunables
`fc242f8`	Oct 31 09:19	F1 and F2 through MusicAgentFacade and AppController per SD03
`c2f9def`	Oct 31 09:25	F3 searchMusic through facade and controller, CLI search
`ac601cf`	Oct 31 09:35	RecommendationEngine and CandidateRetriever for F4

Commit	Toronto time	What it did
`bd3cbc8`	Oct 31 10:18	MusicBrainz genre source behind a GenreEnrichingMusicService decorator (R18)
`1dfac64`	Oct 31 10:46	Exclude known tracks from discover (fix for a live finding)
`457ed81`	Nov 1 09:04	Playlist commands and PlaylistManager with batch rollback and snapshot undo
`3e9a14d`	Nov 1 10:50	F5 create flow with the session states, F11 negotiation and the create command
`cc241b8`	Nov 1 11:13	F6 modify with undo, F7 validate-by-id, F10 explain-by-id and F12 history/restore in facade and controller
`65250b5`	Nov 1 11:34	The CLI as a testable instance with every Stage 1 command and exit statuses
`f70caa3`	Nov 1 11:42	Negotiate an edit that cannot be applied during modify (F11)
`0f8e71d`	Nov 1 17:30	Use each snapshot for one undo so undo never re-applies a change (defect found in PR review)
`13af7f5`	Nov 1 19:24	List the user's playlists and show a playlist
`6c13721`	Nov 1 19:34	Keep a candidate set from search and put it first in the next playlist
`7cbe0b7`	Nov 2 09:20	GUI: shared worker context, login gate, status line, F11 dialog, Why-these expander
`dd964ac`	Nov 2 09:20	GUI: PlaylistView and CreatePlaylistDialog (F5, F6, F7, F10, F11)
`0d69c16`	Nov 2 09:52	GUI: the Create dialog no longer locks a bound slider (the bug that stopped Create from starting)
`15019aa`	Nov 2 20:48	Search puts exact title and artist matches first and removes duplicates
`1bdd3cd`	Nov 2 20:57	Create searches the goal's activity and reviews real tracks against the goal
`1793e5c`	Nov 2 21:09	Delete a playlist after saving a snapshot
`fd5c556`	Nov 2 21:18	Review the picked tracks against the goal and replace rejected ones

All 147 are listed by `git log --no-merges --since="2026-10-27 00:00 -0400" --until="2026-11-03 00:00 -0500" -- src build.gradle settings.gradle` (Toronto is UTC-4 until Nov 1 and UTC-5 after; this command returns 147). The earlier pull requests #1 to #3 were merged into `main`.

Period 3 (November 3-9): in progress, 2 code commits so far

Commit	Toronto time	What it did
`1b18d24`	Nov 3 14:17	Send the goal review only track ids and known genres, never titles or artist names (policy fix, with tests)
`4211b8b`	Nov 3 14:20	Make the prompt-leak guard test reflect over every build method; tidy imports

The other commits on November 3 are documentation and diagram work (the plan, the conformance table, the README, the AI log, the updated UML sources and PNGs, the diagram README) plus one merge, and do not count toward the minimum. At least 3 more meaningful code commits are needed by November 9; the planned GUI redesign and the live goal-fit follow-ups are the expected source. This section will be refreshed before submission.

6.2 Checks performed

- **Automated tests: 846 passing in 51 test classes** (JUnit 5), run as a forced clean `./gradlew build` on November 3 with no keys and no database server. Spotify is a local `HttpServer` in the tests, PostgreSQL is an embedded binary, and the LLM is `MockLLMProvider` under `src/test`.
- **Test first and independent review.** Each task was built test first and then reviewed by a separate agent; the larger pull requests (#1, #2, #4) had a further whole-PR code review. Defects were reproduced with a test before they were fixed.
- **Prompt guard.** `PromptBuilderTest` calls every `PromptBuilder.build*` method by reflection and asserts that no prompt contains a Spotify track title or artist name.

- **Behaviour tests per feature** are named in `docs/STAGE1_CONFORMANCE.md` (for example `CreatePlaylistFlowTest`, `ModifyPlaylistFlowTest`, `PlaylistValidatorTest`, `SpotifyAdapterTest`, `CliAppTest`, `ViewFormatTest`).
- **Stage 1 names.** Every class and member named in the Stage 1 class diagram and traceability table was checked to exist in `src/main` by search.
- **Secrets.** Keys live only in the git-ignored `.env`; `.env.example` holds placeholders; `.spotify-tokens.json` is git-ignored. A search of the tracked files for key-shaped strings found none on November 3. No keys, tokens, Spotify user ids or playlist ids appear in this report.
- **Run against the real Spotify and OpenAI accounts** (from `docs/DAILY_TEST_NOTES.md`; only what is named here was run):

Date	What was run
Oct 31	CLI `profile --refresh`, `profile`, `profile --range long`, `trends`, `search`, `discover` and `discover --novelty 0.9`. Genres for 13 of 20 top artists. 13 of 20 discover results came from the user's own history (then fixed).
Nov 1	CLI `create` (Stage 1's own example: 27 tracks, 8 by new artists, 30%; also a run using a candidate), `validate`, `explain`, `history` and `history <id>`, `modify` ("15 minutes shorter" removed 3 tracks) and `undo` (put them back), `playlists`, `show`, `trends` with all steps, the `Artists:` search section, `candidates`, `prefs set blockedArtists` by name, a blocked track making `validate` fail and `modify` refuse it; `restore` was tried on a test playlist according to the AI log. `feedback` with a bad id (status 2, nothing saved).
Nov 2	GUI, read-only through a throwaway script: login from the saved token, Profile refresh, Trends, Search, Discover, opening a playlist with its badge, Preferences and History; the Create and F11 dialogs were opened for layout and the Create click path was exercised with no Spotify settings so nothing could be written. The author ran Create (a "workout" goal, 6 tracks, finished; songs did not fit the goal) and Modify (6 tracks became 5) in the GUI by hand. Search for a song title returned exact titles in the top 3 in 5 of 5 runs.

- **Not run live** (repeating Section 1.5): goal fit after the November 2 fixes and the November 3 policy change; F11 with a real conflict; `delete`; recreate from a deleted playlist; GUI Undo, Restore, Accept, Reject, Save and candidate buttons pressed by a person; the other preference keys; a successful `feedback` on the real account.

6.3 Important defects found and corrected

Defect	How it was found	Correction
No artist genres at all (Spotify sends none to this app)	First live profile run, Oct 31	MusicBrainz source behind a Decorator, with a cache (R18).
Genre filter returned 0 to 1 tracks	Live search	Genre sent as a plain keyword (R19).
13 of 20 discover results were tracks the listener already knew	Live discover run	Known tracks excluded from every pool.
Mood words matched stock-music titles in search	Live search	Mood words removed and mapped to a genre; ranking weights retuned.
A repeated undo could re-apply a change the listener had reverted	PR #1 review, reproduced with a test	Each undo uses up one snapshot (`undone` flag); failures recover part-way changes.
A retried write could be applied twice; local files shifted positions	PR #1 review	Adds, creates and reorders are not retried on a 5xx; real Spotify positions are used.
A `TRACK_ID` placeholder saved as feedback	Live CLI test	Track id validated in the controller.
After "15 minutes shorter", validation failed against the old target	Live modify	The playlist's target length follows edits and restores.
An explanation said "genres you love" for goal genres	Live explain	Requested genres recorded in the trace; preference claims rejected unless recorded.

Defect	How it was found	Correction
GUI: thumbs labels garbled; bad length entry failed silently; expired login swallowed	Task reviewer and a live read-only run	UTF-8 properties; a message for a bad length; errors surfaced.
GUI: the Create button could never start	PR #4 code review (768 tests had passed)	The dialog no longer disables a bound slider; a write-free probe confirmed the click path.
Candidates were forced into a "workout" playlist and the explanation said they suited the mood	Author's GUI test, Nov 2	Candidates are optional; the explain prompt may only state recorded reasons.
A blocked preference could be overwritten when the store was unreadable	Final review round, Nov 2	Blocking is refused when the stored preferences cannot be read.
The goal review sent track titles and artist names to OpenAI (Spotify Developer Policy section III item 14)	Found by Claude Code while documenting, Nov 3	Ids and MusicBrainz genres only; a reflective guard test (Section 4).

6.4 Remaining limitations

See Section 1.5. In summary: goal fit unproven since the fixes; several paths not yet run live (F11 real conflict, delete, recreate, several GUI controls by hand); the GUI redesign not started and the GUI plain; description-style searches weak; partial genre coverage; the F6 edit-prompt policy question open; Period 1 missed and Period 3 in progress; demonstration videos and cost figures still to be added; the full deterministic test suite and KUMA validation are Stage 3 work.

7 Demonstration index

One row per feature. Each video segment must identify the feature ID and name, show how it is invoked and the required input, run in the actual application with its output, say briefly what it does and where AI is involved, and show an error or alternative case. The recordings together must show both the GUI and the CLI. Video links must be accessible to the instructor and teaching assistants without an access request, and no keys or credentials may appear. **All link and timestamp cells are placeholders for Sarina.** "Run on real accounts so far" repeats Section 6.2 so the author knows which paths are new when recording.

Feature	How to invoke (CLI / GUI)	Input to show	Output and alternative or error case to show	AI involved	Run on real accounts so far	Video link / timestamp
F1 Music Taste Profile	`profile --refresh`, `profile`, `profile --range long` / Profile tab, "Refresh profile"	A time range	Genre bars, top artists and tracks, coverage notice, summary. Alternative: no LLM key gives the labelled computed summary; little history gives a partial notice.	Yes: summary sentence (hybrid)	CLI Oct 31; GUI read-only Nov 2	**[link to be added by Sarina]**
F2 Taste-Change Analysis	`trends` / Trends tab, "Compare my listening"	None	Three ranges, changes between them, narrative. Alternative: a range with no data marked; partial genre coverage gives cautious wording.	Yes: the narrative (hybrid)	CLI Oct 31 and Nov 1; GUI read-only Nov 2	**[link to be added by Sarina]**
F3 Natural-Language Music Search	`search "<text>"`, `candidates add|list|remove|clear` / Search tab, "Add to candidates"	A song title and a description, for example an activity	Ranked tracks (exact title first) and named artists. Alternative: a blank query is refused; no results gives a simpler-query hint; a failed AI reading is labelled "plain keyword search".	Yes: reads the query (hybrid)	CLI Oct 31, Nov 1, Nov 2; GUI read-only Nov 2. Description searches are weak.	**[link to be added by Sarina]**
F4 Personalized Discovery	`discover [--novelty 0.7]` / Discover tab, "Find music"	A novelty setting	New tracks with reasons; none the listener already knows. Alternative: a widening notice; an invalid novelty value rejected.	No	CLI Oct 31 (exclusion fix covered by tests, not re-run live); GUI read-only Nov 2	**[link to be added by Sarina]**
F5 AI Playlist Planning and Generation	`create "<goal>" --minutes 30 --no-explicit --name "..."` / Playlists tab, "Create playlist..."	A goal, a length, flags; show candidate use and `--no-candidates`	Progress through the state names, then the playlist, notices and explanation. Alternative: an unclear goal asks for clarification; no LLM key says so and changes nothing. *Writes a real private playlist.*	Yes: parses the goal, goal review, explanation (hybrid)	CLI Nov 1; GUI by hand Nov 2 (songs did not fit the goal). **Not run since the goal-fit fixes.**	**[link to be added by Sarina]**
F6 Natural-Language Playlist Modification	`playlists`, `show <id>`, `modify <id> "<text>"`, `undo <id>` / Playlists tab, Modify box, Undo	An instruction such as "make it 15 minutes shorter"	Change summary, updated badge, then undo. Alternative: an ambiguous instruction asks a question and changes nothing; a mood-only edit is flagged "Approximate".	Yes: instruction to steps (hybrid)	CLI Nov 1 (modify, undo); GUI Modify by hand Nov 2. **GUI Undo not run by hand.**	**[link to be added by Sarina]**
F7 Playlist Constraint Validation	`validate <id>` / badges on the Playlists tab	A playlist id; a blocked artist or track to make it fail	PASS or FAIL per rule with offending tracks; exit status 1 on failure.	No	CLI Nov 1 (a blocked track made it fail)	**[link to be added by Sarina]**

Feature	How to invoke (CLI / GUI)	Input to show	Output and alternative or error case to show	AI involved	Run on real accounts so far	Video link / timestamp
F8 Preference Memory	`prefs list|get|set` / Preferences tab	For example `explicitPolicy`, `familiarBias`, a blocked artist by name	Saved value and later effect on `discover` or `validate`. Alternative: `familiarBias 2` rejected; unknown artist refused; database down shows safe defaults.	No	CLI Nov 1 (`blockedArtists` only). Other keys and the GUI Save: not run live.	**[link to be added by Sarina]**
F9 Recommendation Feedback	`feedback <trackId> accept|reject` / Accept and Reject buttons	A track id from `discover`	"Recorded" and a changed later ranking. Alternative: a bad id is rejected with status 2 and nothing saved.	No	Bad id only (CLI). A successful verdict and the GUI buttons: not run live.	**[link to be added by Sarina]**
F10 Grounded Recommendation Explanation	`explain <playlistId>` / "Why these?" expander	A playlist id	An explanation using only recorded facts. Alternative: a rejected AI text shows the labelled computed explanation.	Yes: the explanation, checked (AI with fallback)	CLI Nov 1	**[link to be added by Sarina]**
F11 Infeasible-Constraint Negotiation	The prompt during `create` (for example `--minutes 600 --familiar 1`) or `modify` / the conflict dialog	An impossible request	The conflict, numbered options, the applied choice. Alternative: cancel and show that nothing was created or changed on Spotify.	Yes: words the options (hybrid)	**Not run live.** Tests, and the GUI dialog with stand-in text only.	**[link to be added by Sarina]**
F12 Agent History and Snapshot/Restore	`history [<id>]`, `restore <snapshotId>` / History tab, Restore	A saved version of a test playlist	Sessions with plan and result, saved versions, the restored playlist and an undo hint. Alternative: a restore of a deleted playlist offers to recreate it; an unchanged playlist says so.	No	CLI `history` Nov 1; `restore` on a test playlist per the AI log. Recreate and GUI Restore: not run live.	**[link to be added by Sarina]**
New Delete a playlist	`delete <playlistId>` (CLI only)	A test playlist; type its name when asked	The playlist leaves the library and a saved version remains for `restore`. Alternative: a playlist not owned is refused; a wrong typed name does nothing.	No	**Not run live.**	**[link to be added by Sarina]**

The CLI and the GUI are both shown by the rows above (the CLI for every feature, the GUI for F1-F12 except delete). Features that write to Spotify (F5, F6, F12, delete) should be recorded on a test playlist.